

DIGITALNOTES
ON
BLOCK CHAIN TECHNOLOGY
(R20A0522)

B.TECH IV Year – I Sem
(2023-24)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
MALLA REDDY COLLEGE OF ENGINEERING & TECHNOLOGY
(Autonomous Institution – UGC, Govt. of India)

Sponsored by CMR Educational Society

(Affiliated to JNTU, Hyderabad, Approved by AICTE- Accredited by NBA& NAAC– 'A' Grade-ISO9001:2008 Certified)

Maisammaguda, Dhulapally (Post Via Hakimpet), Secunderabad – 500100, Telangana State, India.

Contact Number: 040-23792146/64634237, E-Mail ID: mrcet2004@gmail.com, website: www.mrcet.ac.in



MALLA REDDY COLLEGE OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(R20A0522) BLOCKCHAIN TECHNOLOGY

IV Year B. Tech. CSE–I Sem

L	T/P/ C
3	-/-/- 3

COURSE OBJECTIVES:

This course will enable the students:

1. To enable the student to understand and appreciate, the importance of fundamentals of blockchain technology and application of cryptography in blockchain
2. To gain the awareness about the concepts of various implementations of blockchain technology such as bitcoin, Ethereum, and Hyper ledger

UNIT-I

Introduction to Blockchain Technology – Distributed systems – The history of blockchain – Introduction to blockchain – CAP theorem and blockchain – Benefits and limitations of blockchain – Decentralization using blockchain - Methods of decentralization – Routes to decentralization

UNIT-II

Cryptography in Blockchain: Introduction – cryptographic primitives – Assymmetric cryptography – public and private keys -line interface – Bitcoin improvement proposals (BIPs) – Consensus Algorithms.

UNIT-III

BitCoin Introduction – Transactions – Structure - Transactions types – The structure of a block– The genesis block – The bitcoin network– Wallets and its types– Bitcoin payments– Bitcoin investment and buying and selling bitcoins – Bitcoin installation – Bitcoin programming and the command-line interface – Bitcoin improvement proposals (BIPs).

UNIT-IV

Ethereum - Ethereum block chain- Elements of the Ethereum block chain– Precompiled contracts – Accounts and its types – Block header- Ether – Messages – Mining - Clients and wallets – Trading and investment – The yellow paper - The Ethereum network - Applications developed on Ethereum - Scalability and security issues

UNIT-V

Smart Contract and Hyper ledger – History of Smart Contract – Ricardian contracts - The DAO. Hyper ledger projects – Hyperledger as a protocol – Fabric - Hyperledger Fabric - Sawtoothlake – Corda Architecture.

TEXTBOOKS:

1. Bashir, Mastering Blockchain: Distributed ledger technology, decentralization, and smart contracts explained, 2nd Edition, 2nd Revised edition edition. Birmingham: Packt Publishing, 2018.

REFERENCEBOOKS:

1. A. M. Antonopoulos, Mastering bitcoin, First edition. Sebastopol CA: O'Reilly, 2015.
2. Z. Zheng, S. Xie, H. Dai, X. Chen, and H. Wang, —An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends,|| in 2017 IEEE International Congress on Big Data (BigData Congress), 2017, pp.557–564.

Outcomes:

- Student will be able to understand the fundamentals of blockchain technology.
 - Apply knowledge of implementations of Bitcoin, Ethereum and Hyperledger to develop solutions in the appropriate domains.
-



MALLA REDDY COLLEGE OF ENGINEERING & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDEX

Unit	Contents	Pg.No
I	Introduction to Blockchain Technology	1
	Distributed systems	2
	The history of blockchain	9
	CAP theorem and blockchain	11
	Benefits and limitations of blockchain	12
	Decentralization using blockchain	14
	Methods of decentralization	17
	Routes to decentralization	20
II	Cryptography in Blockchain	26
	Introduction – cryptographic primitives	27
	Assymmetric cryptography	29
	Public and private keys -line interface	31
	Bitcoin improvement proposals (BIPs)	33
	ConsensusAlgorithms	34
III	Introduction to BitCoin	38
	Transactions and Structure	40
	Transactions types	41
	The structure of a block	44
	The bitcoin network– Wallets and its types	46
	Bitcoin payments_ Bitcoin investment and buying and selling Bitcoins	51
	Bitcoin installation: Bitcoin programming and the command-line interface	52
	Bitcoin improvement proposals (BIPs).	57
IV	Ethereum & Elements of the Ethereum block chain	58
	Precompiled contracts	60
	Accounts and its types	61
	Block header	62
	Ether – Messages	64
	Mining - Clients and wallets	67

	Trading and investment – The yellow paper	70
	The Ethereum network - Applications developed on Ethereum	71
	Scalability and security issues	72
V	Smart Contract & History of Smart Contract	74
	Ricardian contracts -The DAO	75
	Hyper ledger projects – Hyperledger as a protocol	76
	Fabric - Hyperledger Fabric	77
	Sawtooth lake – CordaArchitecture.	81

UNIT-I

Blockchain is a buzzword in today's technology and this technology is described as the most disruptive technology of the decade. Thus, Blockchain is used for the secure transference of items like money, contracts, property rights, stocks, and even networks without any requirement of Third Party Intermediaries like Governments, banks, etc. Once the data is stored in the Blockchain it becomes very difficult to manipulate the stored data. A Blockchain is a Network Protocol like SMTP. However, Blockchain cannot be run without the Internet. Blockchain is useful in many areas like Banking, Finance, Healthcare, Insurance, etc.

A blockchain is an open, distributed ledger that can record transactions between two parties efficiently and in a verifiable and permanent way without the need for a central authority.

Key Characteristics:

- **Open:** Anyone can access blockchain.
- **Distributed or Decentralised:** Not under the control of any single authority.
- **Efficient:** Fast and Scalable.
- **Verifiable:** Everyone can check the validity of information because each node maintains a copy of the transactions.
- **Permanent:** Once a transaction is done, it is persistent and can't be altered.

Blockchain can be defined as the Chain of Blocks that contain some specific Information. Thus, a Blockchain is a ledger i.e file that constantly grows and keeps the record of all transactions permanently. This process takes place in a secure, chronological (Chronological means every transaction happens after the previous one) and immutable way. Each time when a block is completed in storing information, a new block is generated.

Distributed Systems:

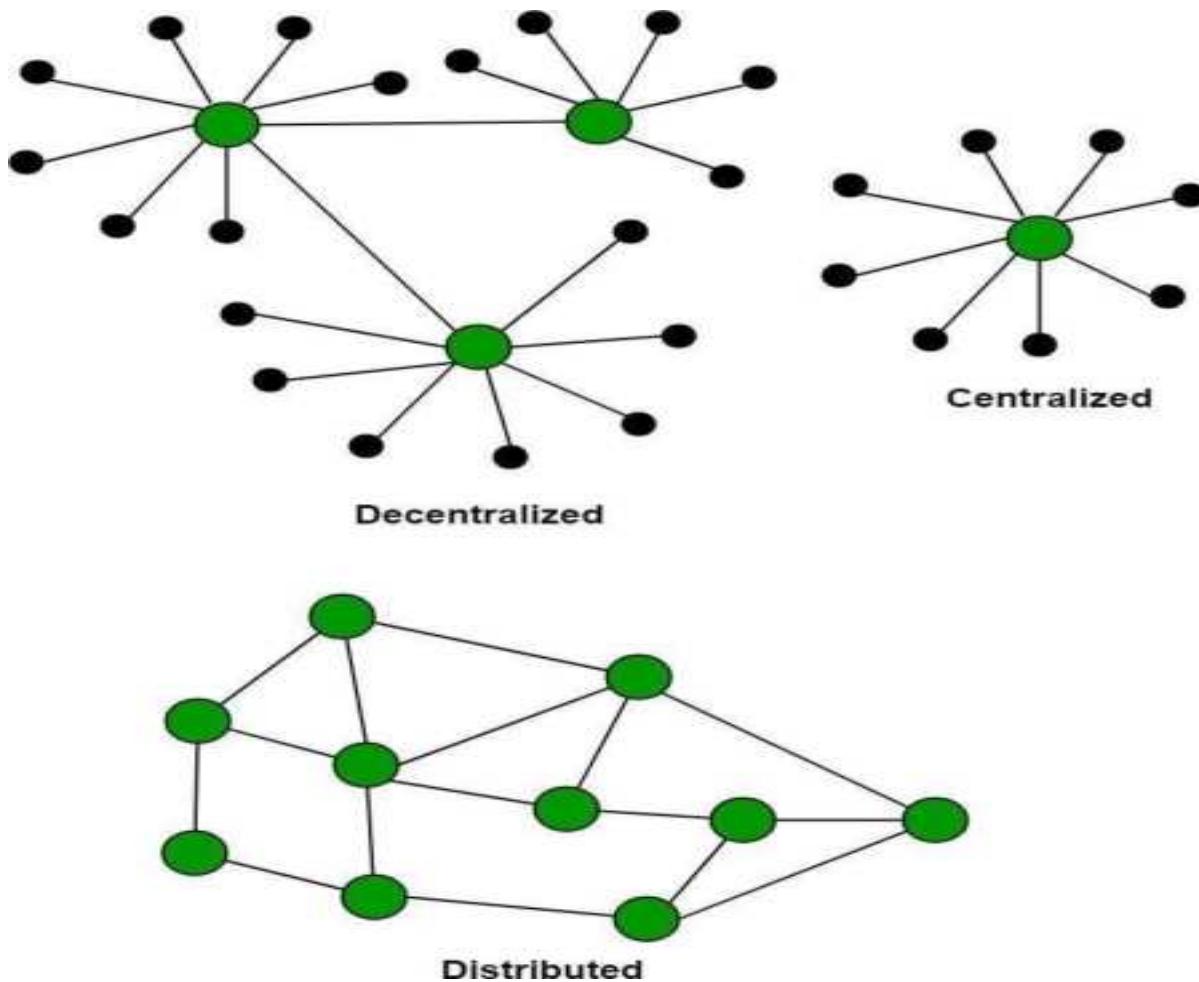
Understanding distributed systems is essential to our understanding blockchain, as blockchain was a distributed system at its core. It is a distributed ledger that can be centralized or decentralized. A blockchain is originally intended to be and is usually used as a decentralized platform. It can be thought of as a system that has properties of the both decentralized and distributed paradigms. It is a decentralized-distributed system.

Distributed systems are a computing paradigm whereby two or more nodes work with each other in a coordinated fashion to achieve a common outcome. It is modeled in such a way that end users see it as a single logical platform. For example, Google's search engine is based on a large distributed system; however, to a user, it looks like a single, coherent platform.

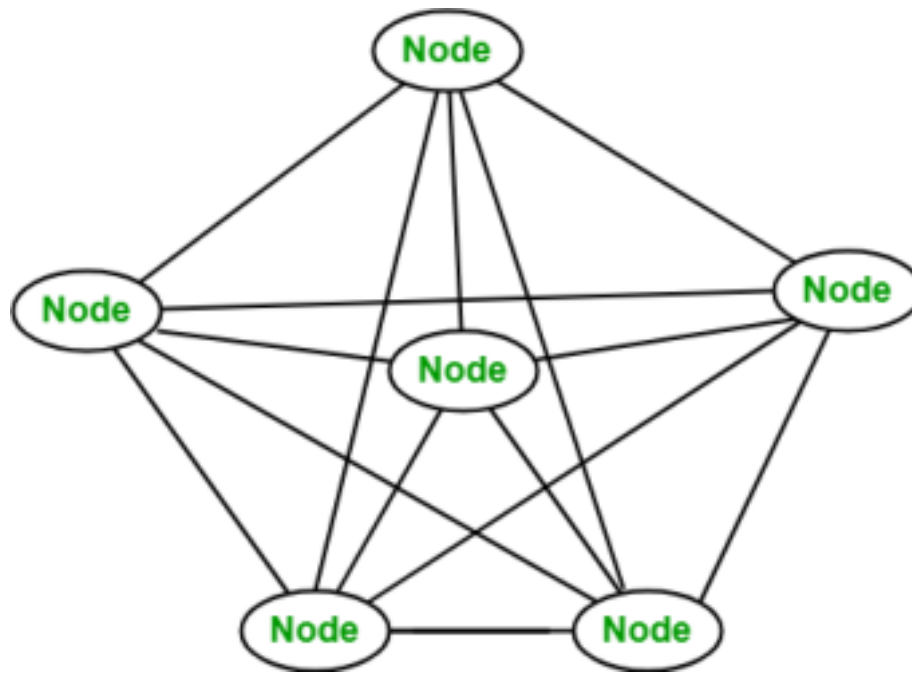
A **node** can be defined as an individual player in a distributed system. All nodes are capable of sending and receiving messages to and from each other. There is no Central Server or System which keeps the data of Blockchain. The data is distributed over Millions of Computers around the world which are connected with the Blockchain. This system allows Notarization of Data as it is present on every Node and is publicly verifiable. A node can be defined as an individual player in a distributed system. All nodes are capable of sending and receiving messages to and from each other.

Nodes can be honest, faulty, or malicious and have their own memory and processor. A node that can exhibit arbitrary behavior is also known as a Byzantine node. This arbitrary behavior can be intentionally malicious, which is detrimental to the operation of the network. Generally, any unexpected behavior of a node on the network can be categorized as Byzantine. This term arbitrarily encompasses any behavior that is unexpected or malicious.

The main challenge in distributed system design is coordination between nodes and fault tolerance. Even if some of the nodes become faulty or network links break, the distributed system should tolerate this and should continue to work flawlessly in order to achieve the desired result. This has been an area of active research for many years and several algorithms and mechanisms have been proposed to overcome these issues.



A network of nodes: A node is a computer connected to the Blockchain Network. Node gets connected with Blockchain using the client. Client helps in validating and propagates transaction on to the Blockchain. When a computer connects to the Blockchain, a copy of the Blockchain data gets downloaded into the system and the node comes in sync with the latest block of data on Blockchain. The Node connected to the Blockchain which helps in the execution of a Transaction in return for an incentive is called Miners.

**Disadvantages of current transaction system:**

- Cash can only be used in low amount transaction locally.
- Huge waiting time in the processing of transactions.
- Need to third party for verification and execution of Transaction make the process complex.
- If the Central Server like Banks is compromised, whole System is affected including the participants.
- Organization doing validation charge high process thus making the process expensive.

Building trust with Blockchain:

Blockchain enhances trust across a business network. It's not that you can't trust those who you conduct business with its that you don't need to when operating on a Blockchain network.

Blockchain builds trust through the following five attributes:

- **Distributed:** The distributed ledger is shared and updated with every incoming transaction among the nodes connected to the Blockchain. All this is done in real-time as there is no central server controlling the data.
- **Secure:** There is no unauthorized access to Blockchain made possible through Permissions and Cryptography.

- **Transparent:** Because every node or participant in Blockchain has a copy of the Blockchain data, they have access to all transaction data. They themselves can verify the identities without the need for mediators.
- **Consensus-based:** All relevant network participants must agree that a transaction is valid. This is achieved through the use of consensus algorithms.
- **Flexible:** Smart Contracts which are executed based on certain conditions can be written into the platform. Blockchain Network can evolve in pace with business processes.

History of Blockchain:

- In 1991, researcher scientists named Stuart Haber and W. Scott Stornetta introduce Blockchain Technology. These scientists wanted some Computational practical Solution for time-stamping the digital documents so that they couldn't be tampered or misdated. So both scientists together developed a system with the help of Cryptography. In this System, the time-stamped documents are stored in a Chain of Blocks.
- After that in 1992, Merkle Trees formed a legal corporation by using a system developed by Stuart Haber and W. Scott Stornetta with some more features. Hence, Blockchain Technology became efficient to store several documents to be collected into one block. Merkle used a Secured Chain of Block which stores multiple data records in a sequence. However, this Technology became unused when Patent came into existence in 2004.
- However, in the same year 2004, Cryptographic activist Hal Finney introduced a system for digital cash known as "Reusable Proof of Work". This step was the game-changer in the history of Blockchain and Cryptography. This System helps others to solve the Double Spending Problem by keeping the ownership of tokens registered on a trusted server.
- After that in 2008, Satoshi Nakamoto conceptualized the concept of "Distributed Blockchain" under his white paper: "A Peer to Peer Electronic Cash System". He modified the model of Merkle Tree and created a system that is more secure and contains the secure history of data exchange. His System follows a peer-to-peer network of time stamping. His system became so useful that Blockchain become the backbone of

Cryptography.

- After that, the evolution of Blockchain is steady and promising and became a need in various fields. Blockchain technology is so secure that the following surprising news will give proof about that. A person named, James Howells was an IT worker in the United Kingdom, he starts mining bitcoins which are part of Blockchain in 2009 and stopped this in 2013. He spends \$17,000 on it and after he stopped he sells the parts of his laptop on eBay and keep the drive with him so that when he needs to work again on bitcoin he will utilize it but while cleaning his house in 2013, he thrashed his drive with garbage and now his bitcoins cost nearly \$127 million. This money now remains unclaimed in the Bitcoin system.

The blockchain is the public ledger of all Bitcoin transactions that have ever been executed. It is constantly growing as miners add new blocks to it (every 10 minutes) to record the most recent transactions. The blocks are added to the blockchain in a linear, chronological order. Each full node (i.e., every computer connected to the Bitcoin network using a client that performs the task of validating and relaying transactions) has a copy of the blockchain, which is downloaded automatically when the miner joins the Bitcoin network. The blockchain has complete information about addresses and balances from the genesis block (the very first transactions ever executed) to the most recently completed block.

Blockchain is the backbone Technology of Digital Cryptocurrency BitCoin. The blockchain is a distributed database of records of all transactions or digital event that have been executed and shared among participating parties. Each transaction verified by the majority of participants of the system. It contains every single record of each transaction. BitCoin is the most popular cryptocurrency an example of the blockchain. Blockchain Technology Records Transaction in Digital Ledger which is distributed over the Network thus making it incorruptible. Anything of value like Land Assets, Cars, etc. can be recorded on Blockchain as a Transaction.

One of the famous use of Blockchain is Bitcoin. The bitcoin is a cryptocurrency and is used to exchange digital assets online. Bitcoin uses cryptographic proof instead of third-party trust for two parties to execute transactions over the internet. Each transaction protects through digital signature.

CAP theorem:

The **CAP theorem**, also known as Brewer's theorem, was introduced by Eric Brewer in 1998 as a conjecture. In 2002, it was proven as a theorem by Seth Gilbert and Nancy Lynch. The theorem states that any distributed system cannot have consistency, availability, and partition tolerance simultaneously:

- **Consistency** is a property that ensures that all nodes in a distributed system have a single, current, and identical copy of the data.

Consistency is achieved using consensus algorithms in order to ensure that all nodes have the same copy of the data. This is also called **state machine replication**. The blockchain is a means for achieving state machine replication.

- **Availability** means that the nodes in the system are up, accessible for use, and are accepting incoming requests and responding with data without any failures as and when required. In other words, data is available at each node and the nodes are responding...

The CAP theorem states that **a distributed database system has to make a tradeoff between Consistency and Availability when a Partition occurs**. A distributed database system is bound to have partitions in a real-world system due to network failure or some other reason.

The CAP Theorem is comprised of three components (hence its name) as they relate to distributed data stores:

Consistency. All reads receive the most recent write or an error.

Availability. All reads contain data, but it might not be the most recent.

Partition tolerance.

The CAP Theorem is comprised of three components (hence its name) as they relate to distributed data stores:

- **Consistency.** All reads receive the most recent write or an error.
- **Availability.** All reads contain data, but it might not be the most recent.
- **Partition tolerance.** The system continues to operate despite network failures (ie; dropped partitions, slow network connections, or unavailable network connections between nodes.)

In normal operations, your data store provides all three functions. But the CAP theorem maintains that when a distributed database experiences a network failure, you can provide either consistency or availability.

It's a tradeoff. All other times, all three can be provided. But, in the event of a network failure, a choice must be made. In the theorem, partition tolerance is a must. The assumption is that the system operates on a distributed data store so the system, by nature, operates with network partitions. Network failures will happen, so to offer any kind of reliable service, partition tolerance is necessary—the P of CAP.

That leaves a decision between the other two, C and A. When a network failure happens, one can choose to guarantee consistency or availability:

- High consistency comes at the cost of lower availability.
- High availability comes at the cost of lower consistency.

Benefits and limitations of blockchain:

Numerous benefits of blockchain technology are being discussed in the industry and proposed by thought leaders around the world in blockchain space. The top 10 benefits are listed and discussed as follows.

Decentralization :

This is a core concept and benefit of blockchain. There is no need for a trusted third party or intermediary to validate transactions; instead a consensus mechanism is used to agree on the validity of transactions.

Transparency and trust :

As blockchains are shared and everyone can see what is on the blockchain, this allows the system to be transparent and as a result trust is established. This is more relevant in scenarios such as the disbursement of funds or benefits where personal discretion should be restricted.

Immutability:

Once the data has been written to the blockchain, it is extremely difficult to change it back. It is not truly immutable but, due to the fact that changing data is extremely difficult and almost impossible, this is seen as a benefit to maintaining an immutable ledger of transactions.

High availability:

As the system is based on thousands of nodes in a peer-to-peer network, and the data is replicated and updated on each and every node, the system becomes highly available. Even if nodes leave the network or become inaccessible, the network as a whole continues to work, thus making it highly available.

Highly secure:

All transactions on a blockchain are cryptographically secured and provide integrity.

Simplification of current paradigms:

The current model in many industries such as finance or health is rather disorganized, wherein multiple entities maintain their own databases and data sharing can become very difficult due to the disparate nature of the systems. But as a blockchain can serve as a single shared ledger among interested parties, this can result in simplifying this model by reducing the complexity of managing the separate systems maintained by each entity.

Faster dealings:

In the financial industry, especially in post-trade settlement functions, blockchain can play a vital role by allowing the quicker settlement of trades as it does not require a lengthy process of verification, reconciliation, and clearance because a single version of agreed upon data is already available on a shared ledger between financial organizations.

Cost saving:

As no third party or clearing houses are required in the blockchain model, this can massively eliminate overhead costs in the form of fees that are paid to clearing houses or trusted third parties.

Decentralization:

Decentralization is a core benefit and service provided by blockchain technology. By design, blockchain is a perfect vehicle for providing a platform that does not need any intermediaries and that can function with many different leaders chosen via consensus mechanisms. This model allows anyone to compete to become the decision-making authority. A consensus mechanism governs this competition, and the most famous method is known as **Proof of Work (PoW)**.

Decentralization is applied in varying degrees from a semi-decentralized model to a fully decentralized one depending on the requirements and circumstances. Decentralization can be viewed from a blockchain perspective as a mechanism that provides a way to remodel existing applications and paradigms, or to build new applications, to give full control to users.

Information and communication technology (ICT) has conventionally been based on a centralized paradigm whereby database or application servers are under the control of a central authority, such as a system administrator. With Bitcoin and the advent of blockchain technology, this model has changed, and now the technology exists to allow anyone to start a decentralized system and operate it with no single point of failure or single trusted authority. It can either be run autonomously or by requiring some human intervention, depending on the type and model of governance used in the decentralized application running on the blockchain.

The following diagram shows the different types of systems that currently exist: central, distributed, and decentralized.

Different types of networks/systems

Centralized systems are conventional (client-server) IT systems in which there is a single authority that controls the system, and who is solely in charge of all operations on the system. All users of a centralized system are dependent on a single source of service. The majority of online service providers, including Google, Amazon, eBay, and Apple's App Store, use this conventional model to deliver services.

In a **distributed system**, data and computation are spread across multiple nodes in the network. Sometimes, this term is confused with *parallel computing*. While there is some overlap in the definition, the main difference between these systems is that in a parallel computing system, computation is performed by all nodes simultaneously in order to achieve the result; for example, parallel computing platforms are used in weather research and forecasting, simulation, and financial modeling. On the other hand, in a distributed system,

computation may not happen in parallel and data is replicated across multiple nodes that users view as a single, coherent system. Variations of both of these models are used to achieve fault tolerance and speed. In the parallel system model, there is still a central authority that has control over all nodes and governs processing. This means that the system is still centralized in nature.

The critical difference between a decentralized system and distributed system is that in a distributed system, there is still a central authority that governs the entire system, whereas in a decentralized system, no such authority exists.

A **decentralized system** is a type of network where nodes are not dependent on a single master node; instead, control is distributed among many nodes. This is analogous to a model where each department in an organization is in charge of its own database server, thus taking away the power from the central server and distributing it to the sub-departments, who manage their own databases.

A significant innovation in the decentralized paradigm that has given rise to this new era of decentralization of applications is **decentralized consensus**. This mechanism came into play with Bitcoin, and it enables a user to agree on something via a consensus algorithm without the need for a central, trusted third party, intermediary, or service provider.

We can also now view the different types of networks shown earlier from a different perspective, where we highlight the controlling authority of these networks as a symbolic hand, as shown in the following diagram. This model provides a clearer understanding of the differences between these networks from a decentralization point of view,

Different types of networks/systems depicting decentralization from a modern perspective

In the middle we have distributed systems, where we still have a central controller but the system comprises many dispersed nodes. On the right-hand side, notice that there is no hand/controller controlling the networks.

This is the key difference between decentralized and distributed networks. A decentralized system may look like a distributed system from a topological point of view, but it doesn't have a central authority that controls the network.

A traditional distributed system comprises many servers performing different roles

The following diagram shows a decentralized system (based on blockchain) where an exact replica of the applications and data is maintained across the entire network on each participating node:

A comparison between centralized and decentralized systems (networks/applications) is shown in the following table:

Feature	Centralized	Decentralized
Ownership	Service provider	All users
Architecture	Client/server	Distributed, different topologies
Security	Basic	More secure
High availability	No	Yes
Fault tolerance	Basic, single point of failure	Highly tolerant, as service is replicated
Collusion resistance	Basic, because it's under the control of a group or even single individual	Highly resistant, as consensus algorithms ensure defense against adversaries
Application architecture	Single application	Application replicated across all nodes on the network
Trust	Consumers have to trust the service provider	No mutual trust required
Cost for consumer	Higher	Lower

The comparison in the table only covers some main features and is not an exhaustive list of all features. There may be other features of interest that can be compared too, but this list should provide a good level of comparison.

Now we will discuss what methods can be used to achieve decentralization.

Methods of Decentralization:

Two methods can be used to achieve decentralization: disintermediation and competition. These methods will be discussed in detail in the sections that follow.

The concept of **disintermediation** can be explained with the aid of an example. Imagine that you want to send money to a friend in another country. You go to a bank, which, for a fee, will transfer your money to the bank in that country. In this case, the bank maintains a central database that is updated, confirming that you have sent the money. With blockchain technology, it is possible to send this money directly to your friend without the need for a bank. All you need is the address of your friend on the blockchain. This way, the intermediary (that is, the bank) is no longer required, and decentralization is achieved by disintermediation. It is debatable, however, how practical decentralization through disintermediation is in the financial sector due to the massive regulatory and compliance requirements. Nevertheless, this model can be used not only in finance but in many other industries as well, such as health, law, and the public sector. In the health industry, where patients, instead of relying on a trusted third party (such as the hospital record system) can be in full control of their own identity and their data that they can share directly with only those entities that they trust. As a general solution, blockchain can serve as a decentralized health record management system where health records can be exchanged securely and directly between different entities (hospitals, pharmaceutical companies, patients) globally without any central authority.

Contest-driven decentralization:

In the method involving **competition**, different service providers compete with each other in order to be selected for the provision of services by the system. This paradigm does not achieve complete decentralization. However, to a certain degree, it ensures that an intermediary or service provider is not monopolizing the service. In the context of blockchain technology, a system can be envisioned in which smart contracts can choose an external data provider from a large number of providers based on their reputation, previous score, reviews, and quality of service.

This method will not result in full decentralization, but it allows smart contracts to make a free choice based on the criteria just mentioned. This way, an environment of competition is cultivated among service providers where they compete with each other to become the data provider of choice.

In the following diagram, varying levels of decentralization are shown. On the left side, the conventional approach is shown where a central system is in control; on the right side, complete disintermediation is achieved, as intermediaries are entirely removed. Competing intermediaries or service providers are shown in the center. At that level, intermediaries or service providers are selected based on reputation or voting, thus achieving partial decentralization:

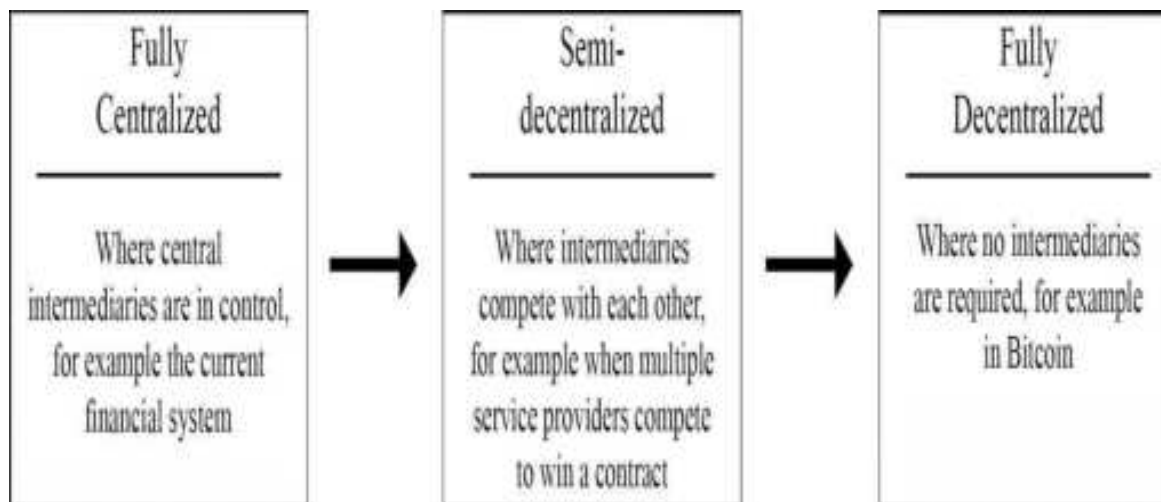


Figure : Scale of decentralization

There are many benefits of decentralization, including transparency, efficiency, cost saving, development of trusted ecosystems, and in some cases privacy and anonymity. Some challenges, such as security requirements, software bugs, and human error, need to be examined thoroughly.

This view raises some fundamental questions. Is a blockchain really needed? When is a blockchain required? In what circumstances is blockchain preferable to traditional databases? To answer these questions, go through the simple set of questions presented below:

Question	Yes/No	Recommended solution
Is high data throughput required?	Yes	Use a traditional database.

	No	A central database might still be useful if other requirements are met. For example, if users trust each other, then perhaps there is no need for a blockchain. However, if they don't or trust cannot be established for any reason, blockchain can be helpful.

Are updates centrally controlled?	Yes	Use a traditional database.
	No	You may investigate how a public/private blockchain can help.
Do users trust each other?	Yes	Use a traditional database.
	No	Use a public blockchain.
Are users anonymous?	Yes	Use a public blockchain.
	No	Use a private blockchain.
Is consensus required to be maintained within a consortium?	Yes	Use a private blockchain.
	No	Use a public blockchain.
Is strict data immutability required?	Yes	Use a blockchain.
	No	Use a central/traditional database.

Answering all of these questions can help you decide whether or not a blockchain is required or suitable for solving the problem. Beyond the questions posed in this model, there are many other issues to consider, such as latency, choice of consensus mechanisms, whether consensus is required or not, and where consensus is going to be achieved. If consensus is maintained internally by a consortium, then a private blockchain should be used; otherwise, if consensus is required publicly among multiple entities, then a public blockchain solution should be considered. Other aspects, such as immutability, should also be considered when deciding whether to use a blockchain or a traditional database. If strict data immutability is required, then a public blockchain should be used; otherwise, a central database may be an option.

As blockchain technology matures, there will be more questions raised regarding this selection model. For now, however, this set of questions is sufficient for deciding whether a blockchain-based solution is suitable or not.

Now we understand different methods of decentralization and have looked at how to decide whether a blockchain is required or not in a particular scenario. Let's now look at the process of decentralization, that is, how we can take an existing system and decentralize it.

Routes to decentralization:

There are systems that pre-date blockchain and Bitcoin, including BitTorrent and the Gnutella file-sharing system, which to a certain degree could be classified as decentralized, but due to a lack of any incentivization mechanism, participation from the community gradually decreased. There wasn't any incentive to keep the users interested in participating in the growth of the network. With the advent of blockchain technology, many initiatives are being taken to leverage this new technology to achieve decentralization. The Bitcoin blockchain is typically the first choice for many, as it has proven to be the most resilient and secure blockchain and has a market cap of nearly \$166 billion at the time of writing. Alternatively, other blockchains, such as Ethereum, serve as the tool of choice for many developers for building decentralized applications. Compared to Bitcoin, Ethereum has become a more prominent choice because of the flexibility it allows for programming any business logic into the blockchain by using **smart contracts**.

How to decentralize

The framework raises four questions whose answers provide a clear understanding of how a system can be decentralized:

1. What is being decentralized?
2. What level of decentralization is required?
3. What blockchain is used?
4. What security mechanism is used?

The first question simply asks you to identify what system is being decentralized. This can be any system, such as an identity system or a trading system.

The second question asks you to specify the level of decentralization required by examining the scale of decentralization, as discussed earlier. It can be full disintermediation or partial disintermediation.

The third question asks developers to determine which blockchain is suitable for a particular application. It can be Bitcoin blockchain, Ethereum blockchain, or any other blockchain that is deemed fit for the specific application.

Finally, a fundamental question that needs to be addressed is how the security of a decentralized system will be guaranteed. For example, the security mechanism can be atomicity-based, where either the transaction executes in full or does not execute at all. This deterministic approach ensures the integrity of the system. Other mechanisms may include one based on reputation, which allows for varying degrees of trust in a system.

In the following section, let's evaluate a money transfer system as an example of an application selected to be decentralized.

Decentralization framework example:

The four questions discussed previously are used to evaluate the decentralization requirements of this application. The answers to these questions are as follows:

1. Money transfer system
2. Disintermediation
3. Bitcoin
4. Atomicity

The responses indicate that the money transfer system can be decentralized by removing the intermediary, implemented on the Bitcoin blockchain, and that a security guarantee will be provided via atomicity. Atomicity will ensure that transactions execute successfully in full or do not execute at all. We have chosen the Bitcoin blockchain because it is the longest established blockchain and has stood the test of time.

Similarly, this framework can be used for any other system that needs to be evaluated in terms of decentralization. The answers to these four simple questions help clarify what approach to take to decentralize the system.

To achieve complete decentralization, it is necessary that the environment around the blockchain also be decentralized. We'll look at the full ecosystem of decentralization next.

The blockchain is a distributed ledger that runs on top of conventional systems. These elements include storage, communication, and computation.

Storage

Data can be stored directly in a blockchain, and with this fact it achieves decentralization. However, a significant disadvantage of this approach is that a blockchain is not suitable for storing large amounts of data by design. It can store simple transactions and some arbitrary data, but it is certainly not suitable for storing images or large blobs of data, as is the case with traditional database systems.

A better alternative for storing data is to use **distributed hash tables (DHTs)**. DHTs were used initially in peer-to-peer file sharing software, such as BitTorrent, Napster, Kazaa, and Gnutella. DHT research was made popular by the CAN, Chord, Pastry, and Tapestry projects. BitTorrent is the most scalable and fastest network, but the issue with BitTorrent and the others is that there is no incentive for users to keep the files indefinitely. Users generally don't keep files permanently, and if nodes that have data still required by someone leave the network, there is no way to retrieve it except by having the required nodes rejoin the network so that the files once again become available.

Two primary requirements here are high availability and link stability, which means that data should be available when required and network links also should always be accessible. **Inter-Planetary File System (IPFS)** by Juan Benet possesses both of these properties, and its vision is to provide a decentralized World Wide Web by replacing the HTTP protocol. IPFS uses Kademlia DHT and Merkle **Directed Acyclic Graphs (DAGs)** to provide storage and searching functionality, respectively.

The incentive mechanism for storing data is based on a protocol known as Filecoin, which pays incentives to nodes that store data using the Bitswap mechanism. The Bitswap mechanism lets nodes keep a simple ledger of bytes sent or bytes received in a one-to-one relationship. Also, a Git-based version control mechanism is used in IPFS to provide structure and control over the versioning of data.

There are other alternatives for data storage, such as Ethereum Swarm, Storj, and MaidSafe. Ethereum has its own decentralized and distributed ecosystem that uses Swarm for storage and the Whisper protocol for communication. MaidSafe aims to provide a decentralized World Wide Web. All of these projects are discussed later in this book in greater detail.

BigChainDB is another storage layer decentralization project aimed at providing a scalable, fast, and linearly scalable decentralized database as opposed to a traditional filesystem. BigChainDB complements decentralized processing platforms and filesystems such as Ethereum and IPFS.

Communication

The Internet (the communication layer in blockchain) is considered to be decentralized. This belief is correct to some extent, as the original vision of the Internet was to develop a decentralized communications system. Services such as email and online storage are now all based on a paradigm where the service provider is in control, and users trust such providers to grant them access to the service as requested. This model is based on the unconditional trust of a central authority (the service provider) where users are not in control of their data. Even user passwords are stored on trusted third-party systems.

Thus, there is a need to provide control to individual users in such a way that access to their data is guaranteed and is not dependent on a single third party.

Access to the Internet (the communication layer) is based on **Internet Service Providers (ISPs)** who act as a central hub for Internet users. If the ISP is shut down for any reason, then no communication is possible with this model.

An alternative is to use **mesh networks**. Even though they are limited in functionality when compared to the Internet, they still provide a decentralized alternative where nodes can talk directly to each other without a central hub such as an ISP.

Now imagine a network that allows users to be in control of their communication; no one can shut it down for any reason. This could be the next step toward decentralizing communication networks in the blockchain ecosystem. It must be noted that this model may only be vital in a jurisdiction where the Internet is censored and controlled by the government.

As mentioned earlier, the original vision of the Internet was to build a decentralized network; however, over the years, with the advent of large-scale service providers such as Google, Amazon, and eBay, control is shifting toward these big players. For example, email is a decentralized system at its core; that is, anyone can run an email server with minimal effort and can start sending and receiving emails. There are better alternatives available. For example, Gmail and Outlook already provide managed services for end users, so there is a natural inclination toward selecting one of these large centralized services as they are more convenient and free to use. This is one example that shows how the Internet has moved toward centralization.

Free services, however, are offered at the cost of exposing valuable personal data, and many users are unaware of this fact. Blockchain has revived the vision of decentralization across the world, and now concerted efforts are being made to harness this technology and take advantage of the benefits that it can provide.

Computing power and decentralization:

Decentralization of computing or processing power is achieved by a blockchain technology such as Ethereum, where smart contracts with embedded business logic can run on the blockchain network. Other blockchain technologies also provide similar processing-layer platforms, where business logic can run over the network in a decentralized manner.

The following diagram shows an overview of a decentralized ecosystem. In the bottom layer, the Internet or mesh networks provide a decentralized communication layer. In the next layer up, a storage layer uses technologies such as IPFS and BigChainDB to enable decentralization. Finally, in the next level up, you can see that the blockchain serves as a decentralized processing (computation) layer. Blockchain can, in a limited way, provide a storage layer too, but that severely hampers the speed and capacity of the system. Therefore, other solutions such as IPFS and BigChainDB are more suitable for storing large amounts of data in a decentralized way. The Identity and Wealth layers are shown at the top level. Identity on the Internet is a vast topic, and systems such as bitAuth and OpenID provide authentication and identification services with varying degrees of decentralization and security assumptions:



Decentralized ecosystem

The blockchain is capable of providing solutions to various issues relating to decentralization. A concept relevant to identity known as **Zooko's Triangle** requires that the naming system in a network protocol is secure,

decentralized, and able to provide human-meaningful and memorable names to the users. Conjecture has it that a system can have only two of these properties simultaneously.

Nevertheless, with the advent of blockchain in the form of **Namecoin**, this problem was resolved. It is now possible to achieve security, decentralization, and human-meaningful names with the Namecoin blockchain. However, this is not a panacea, and it comes with many challenges, such as reliance on users to store and maintain private keys securely. This opens up other general questions about the suitability of decentralization to a particular problem.

Decentralization may not be appropriate for every scenario. Centralized systems with well-established reputations tend to work better in many cases. For example, email platforms from reputable companies such as Google or Microsoft would provide a better service than a scenario where individual email servers are hosted by users on the Internet.

There are many projects underway that are developing solutions for a more comprehensive distributed blockchain system. For example, Swarm and Whisper are developed to provide decentralized storage and communication for Ethereum.

UNIT-II

Cryptography in Blockchain:

The Blockchain is the invention that allows digitally generated information to be allocated without being copied. Blockchain Technology is the heart of the new internet i.e. digital currency, Bitcoin and any other online transaction. Tech experts found a big potential in this technology. “Blockchain is an incorruptible digital ledger of economic transaction that can be programmed to record not just financial transactions but virtually everything of value.” In plain layout, the data is not owned by any single computer but by a chain of computers so that the blocks of data are secured and bound to each other using chain, that technology is known as Blockchain technology. There is no transaction cost due to Blockchain, in Layman language Blockchain is a process to pass information or data from A to B in a safe and automated manner.

Cryptocurrency works on the principle of Blockchain Technology, that is why, Blockchain is the most trending item of current era, due to its secure nature cryptocurrency is widely accepted. Its value is increasing day by day. Many oil industries, IBM Technologies, Mercedes Benz, Swiss Bank, Samsung, and even Google is planning to launch their own cryptocurrency in 2019 for safe and secure transactions. Now, this technology is disrupting almost every marketshare due to its popularity and demand in the world.

Satoshi Nakamoto introduced the concept of Blockchain in 2008 in the form of cryptocurrency Bitcoin. Its function is to allow users to secure and control their monetary values so that no third party like government or banks would be able to access or control it. It is a process to carry everyone to the highest grade of liability. Three technologies work behind the Blockchain Technology-

- Private Key Cryptography
- Peer 2 Peer Network
- Blockchain's Protocol Program

Private Key Cryptography

- Peer 2 Peer Network
- Blockchain's Protocol Program

Introduction – cryptographic primitives:

Private Key Cryptography

- Peer 2 Peer Network
- BlockChain's Protocol Program

Blockchain, as one of the crypto-intensive creatures, has become a very hot topic recently. Although many surveys have recently been dedicated to the security and privacy issues of blockchains, there still lacks a systematic examination on the cryptographic primitives in blockchains.

Since its introduction in the early 1980s (Chaum, 1982), the design of e-cash has always been one of the main research topics in the field of cryptography. However, the one without any trusted third party remained an open problem till Bitcoin (Nakamoto) launched in 2009. Due to its decentralization, unforgeability, double-spending resistance and pseudonymity, this brand new e-cash system has brought a remarkable culmination of cryptocurrency research and its applications. Based on its main framework, many new cryptocurrencies including decentralized (such as Litecoin), Nxtcoin (Nxt)) and centralized ones (such as RScoin (Danezis and Meiklejohn, 2016)) have been proposed. The market value of these cryptocurrencies has increased more than 30 times during 2017 (from about \$17 billion on 1st Jan. to \$591 billion on 31st Dec.) (Coinmarketcap). As the core technology behind Bitcoin, the blockchain has demonstrated its capability of innovation and infiltration in many domains, including finance, insurance, industry, healthcare, agriculture and so on.

There are many recent surveys have been dedicated to the security and privacy issues of blockchains .

classify cryptographic primitives in blockchains into two categories: primary and optional. The former category includes cryptographic hashes and standard digital signatures that are essential for ensuring the blockchain as a globe ledger with tamper-proof, public verifiability and achievable consensus. While the latter category, mainly used for enhancing the privacy and anonymity of blockchain-based transactions, covers some special signatures (such as ring signatures), commitments, accumulators, zero-knowledge proofs and so on.

Special signature primitives for blockchains: To enhance the privacy and anonymity of transactions, some advanced signature primitives such as ring signature and multi-signature are also widely applied in blockchains.

1. Ring signatures :

Anonymity is always required in information systems (Shen et al., 2018), especially in the e-cash system. However, Bitcoin can only provide pseudonymity due to the linkability of transactions. Therefore, many new alternative cryptocurrencies have been proposed to address this problem. From a perspective of cryptography, there are many kinds of signatures for achieving anonymity, such as blind signature (Chaum, 1982), ring signature (Rivest et al., 2001), group signature (Chaum and van Heyst, 1991) and DC-nets (Chaum, 1988). However, only ring signature and its variants have been used in blockchains for anonymity.

2. One-time (ring) signatures:

Lamport in 1979 (Lamport, 1979) proposed the concept of one-time signature (OTS), where the signing key can be used *securely but only once*, and the signing key would be revealed if it is used twice or more. OTS is frequently used as a building block in constructions of encryptions and authenticated keyagreements.

3. Borromean (ring) signatures:

Another interesting primitive related to ring signature and blockchain is the so-called Borromean (ring) signature (BRS), proposed by Maxwell and Poelstra in 2015 (Maxwell and Poelstra, 2015). Poelstra (Poelstra, 2017) claimed that BRS is now used in Elements (Element, 2015), Liquid (Liquid) and Monero.

4. Multi-signatures:

The primitive of multi-signature allows a single signature to work as several ordinary signatures on the same message. One of the critical requirements of multi-signature is that the single signature has the same size as one regular signature.

Assymmetric cryptography:

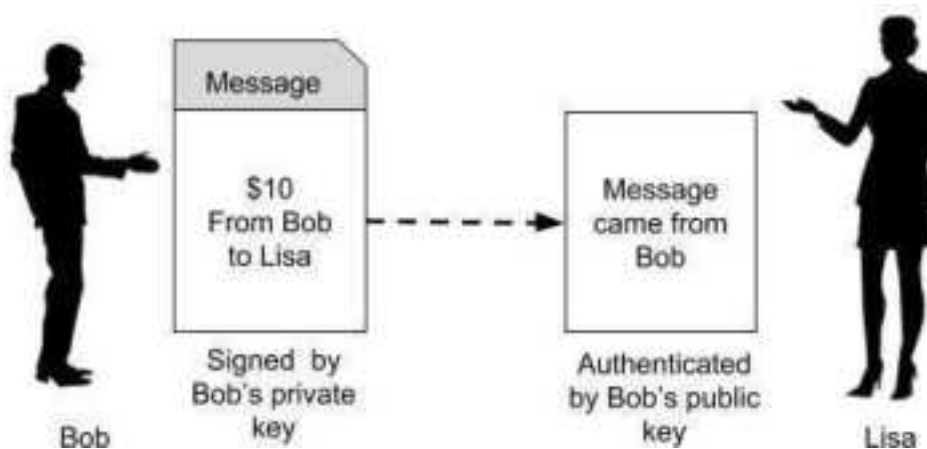
Public Key Cryptography or in short PKI is also known as asymmetric cryptography. It uses two pairs of keys - public and private. A key is a some long binary number. The public key is distributed worldwide and is truly public as its name suggests. The private key is to be strictly held private and one should never lose it.

In case of Bitcoin, if you ever lose the private key to your Bitcoin wallet, the entire contents of your wallets would be instantly vulnerable to theft and before you know it, all your money (the contents of your wallet) would be gone with no mechanism in the system to trace out who stole it - that is the anonymity in the system that I mentioned earlier.

The PKI accomplishes two functions - authentication and the message privacy through encryption/decryption mechanism. I will now explain both these functions

Authentication

When the two parties exchange messages, it is important to establish a trust between the sender and the receiver. Especially, the receiver must trust the source of message. Going to our earlier scenario (depicted in Figure 1) of Bob sending some money to Lisa for purchasing of some goods from her, let us see how the PKI builds this trust between Bob and Lisa. Look at below image



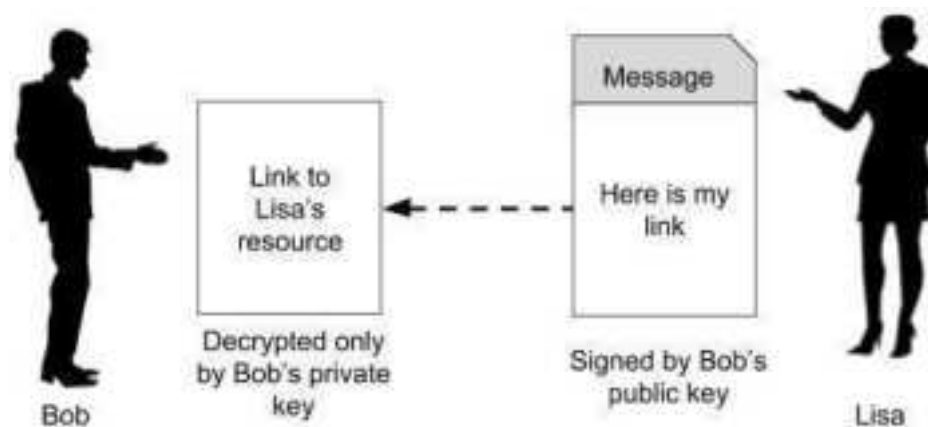
In the first place, if Bob wants to send some money to Lisa, he has to create a private/public key of its own. Note that both keys are always paired together and you can not mix the private and public keys of different individuals or different instances.

Now, Bob says that he is sending \$10 to Lisa. So he creates a message (a plain-text message) containing Bob's (sender) public key, Lisa's (receiver) public key, and the amount (\$10).

The purpose of this remittance such as "I want to buy pumpkin from you" is also added into the message. The entire message is now signed using Bob's private key. When Lisa receives this message, she will use the signature verification algorithm of PKI and Bob's public key to ensure that the message indeed originated from Bob. How the PKI works is beyond the scope of this tutorial. The interested reader is referred to this site for a more detailed discussion on PKI. This establishes the authenticity of the message originator. Now, let us look at the message privacy

Message Privacy:

Now, as Lisa has received her payment, she wants to send the link to her ebook which Bob wants to buy. So Lisa would create a message and send it to Bob as shown in image



Public and private keys -line interface:

In order to understand public key cryptography, the first concept that needs to be looked at is the idea of public and private keys.

A private key, as the names suggests, is basically a randomly generated number that is kept secret and held privately by the users. Private key needs to be protected and no unauthorized access should be granted to that key; otherwise, the whole scheme of public key cryptography will be jeopardized as this is the key that is used to decrypt messages. Private keys can be of various lengths depending upon the type and class of algorithms used. For example, in RSA, typically, a key of 1024-bit or 2048-bits is used. 1024-bit key size is no longer considered secure and at least 2048 bit is recommended to be used in practice.

A public key is the public part of the private-public key pair. A public key is available publicly and published by the private key owner. Anyone who would then like to send the publisher of the public key an encrypted message.

The Lisa creates a message such as “Here is the link to my ebook which you had requested”, signs it with Bob’s public key that she has received in Bob’s request message and also encrypts the message using some secret key which is shared between the two during HTTPS handshake.

Now, Lisa is sure that only Bob can decode the message using the private key that is held by Bob alone. Also, somebody intercepting the message would not be able to recover its contents because the contents are encrypted by a secret key held only by Bob and Alice. This guarantees to Lisa that access to her ebook is granted only to Bob.

Having seen both the features, Authentication and Message Privacy, implied by PKI, let us move ahead to see how Bitcoin makes use of PKI to secure the public ledger .

Public And Private Keys:

Bitcoin, as well as all other major cryptocurrencies that came after it, is built upon public-key cryptography, a cryptographic system that uses pairs of keys: public keys, which are publicly known and essential for identification, and private keys, which are kept secret and are used for authentication and encryption.

Major cryptocurrencies like Bitcoin, Ethereum, and Bitcoin Cash function using three fundamental pieces of information: the address, associated with a balance and used for sending and receiving funds, and the address' corresponding public and private keys. The generation of a bitcoin address begins with the generation of a private key. From there, its corresponding public key can be derived using a known algorithm. The address, which can then be used in transactions, is a shorter, representative form of the public key.

The private key is what grants a cryptocurrency user ownership of the funds on a given address. The Blockchain wallet automatically generates and stores private keys for you. When you send from a Blockchain wallet, the software signs the transaction with your private key (without actually disclosing it), which indicates to the entire network that you have the authority to transfer the funds on the address you're sending from.

The security of this system comes from the one-way street that is getting from the private key to the public address. It is not possible to derive the public key from the address; likewise, it is impossible to derive the private key from the public key.

In the Blockchain.com Wallet, your 12-word Secret Private Key Recovery Phrase is a seed of all the private keys of all the addresses generated within the wallet. This is what allows you to restore access to your funds even if you lose access to your original wallet. Using the recovery phrase will allow you to recover your crypto.

Bitcoin improvement proposals (BIPs):

A Bitcoin Improvement Proposal (BIP) is a design document for introducing features or information to Bitcoin. This is the standard way of communicating ideas since Bitcoin has no formal structure.

The first BIP (BIP 0001) was submitted by Amir Taaki on 2011-08-19 and described what a BIP is.

Types

There are three types of BIPs:

- Standards Track BIPs - Changes to the network protocol, block or transaction validation, or anything affecting interoperability.
- Informational BIPs - Design issues, general guidelines. This type of BIP is NOT for proposing new features and do not represent community consensus
- Process BIPs - Describes or proposes a change in process. Similar to Standards BIPs but apply outside the Bitcoin protocol.

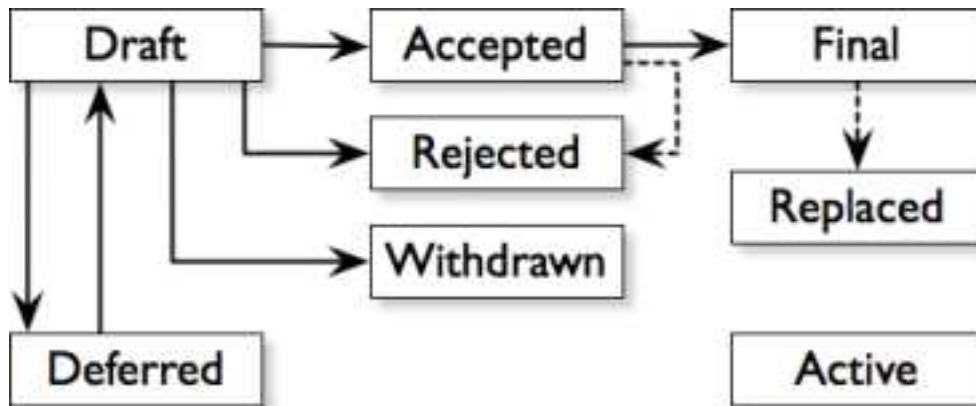
Layers

BIP 0123 established four layers for Standards BIPs:

1. Consensus
2. Peer Services
3. API/RPC
4. Applications

Workflow:

As described in BIP 0001 the workflow of a BIP is as follows:

**ConsensusAlgorithms:**

We know that Blockchain is a distributed decentralized network that provides immutability, privacy, security, and transparency. There is no central authority present to validate and verify the transactions, yet every transaction in the Blockchain is considered to be completely secured and verified. This is possible only because of the presence of the consensus protocol which is a core part of any Blockchain network.

A consensus algorithm is a procedure through which all the peers of the Blockchain network reach a common agreement about the present state of the distributed ledger. In this way, consensus algorithms achieve reliability in the Blockchain network and establish trust between unknown peers in a distributed computing environment. Essentially, the consensus protocol makes sure that every new block that is added to the Blockchain is the one and only version of the truth that is agreed upon by all the nodes in the Blockchain.

The Blockchain consensus protocol consists of some specific objectives such as coming to an agreement, collaboration, co-operation, equal rights to every node, and mandatory participation of each node in the

consensus process. Thus, a consensus algorithm aims at finding a common agreement that is a win for the entire network.

Now, we will discuss various consensus algorithms and how they work.

Proof of Work (PoW):

This consensus algorithm is used to select a miner for the next block generation. Bitcoin uses this PoW consensus algorithm. The central idea behind this algorithm is to solve a complex mathematical puzzle and easily give out a solution. This mathematical puzzle requires a lot of computational power and thus, the node who solves the puzzle as soon as possible gets to mine the next block. For more details on PoW, please read Proof of Work (PoW) Consensus

Practical Byzantine Fault Tolerance (PBFT):

Please refer to the existing article on practical Byzantine Fault Tolerance (PBFT).

Proof of Stake (PoS):

This is the most common alternative to PoW. Ethereum has shifted from PoW to PoS consensus. In this type of consensus algorithm, instead of investing in expensive hardware to solve a complex puzzle, validators invest in the coins of the system by locking up some of their coins as stake. After that, all the validators will start validating the blocks. Validators will validate blocks by placing a bet on it if they discover a block which they think can be added to the chain. Based on the actual blocks added in the Blockchain, all the validators get a reward proportionate to their bets and their stake increase accordingly.

In the end, a validator is chosen to generate a new block based on their economic stake in the network. Thus, PoS encourages validators through an incentive mechanism to reach to an agreement.

Proof of Burn (PoB):

With PoB, instead of investing into expensive hardware equipment, validators 'burn' coins by sending them to an address from where they are irretrievable. By committing the coins to an unreachable address, validators earn a privilege to mine on the system based on a random selection process. Thus, burning coins here means that validators have a long-term commitment in exchange for their short-term loss.

Depending on how the PoB is implemented, miners may burn the native currency of the Blockchain application or the currency of an alternative chain, such as bitcoin. While PoB is an interesting alternative to PoW, the protocol still wastes resources needlessly. And it is also questioned that mining power simply goes to those who are willing to burn more money.

Proof of Capacity:

In the Proof of Capacity consensus, validators are supposed to invest their hard drive space instead of investing in expensive hardware or burning coins. The more hard drive space validators have, the better are their chances of getting selected for mining the next block and earning the block reward.

Proof of Elapsed Time:

PoET is one of the fairest consensus algorithms which chooses the next block using fair means only. It is widely used in permissioned Blockchain networks. In this algorithm, every validator on the network gets a fair chance to create their own block. All the nodes do so by waiting for random amount of time, adding a proof of their wait in the block. The created blocks are broadcasted to the network for others consideration. The winner is the validator which has least timer value in the proof part. The block from the winning validator node gets appended to the Blockchain. There are additional checks in the algorithm to stop nodes from always winning the election, stop nodes from generating a lowest timer value.

There also exist other consensus algorithms like Proof of Activity, Proof of Weight, Proof of Importance, Leased Proof of Stake, etc. It is therefore important to wisely choose one as per the business network requirement because Blockchain networks cannot function properly without the consensus algorithms to verify each and every transaction that is being committed.

UNIT – III

INTRODUCTION TO BITCOIN:

Bitcoin is the first application of the blockchain technology. Bitcoin has started a revolution with the introduction of the very first fully decentralized digital currency, and one that has proven to be extremely secure and stable. This has also sparked a great interest in academic and industrial research and introduced many new research areas.

Since its introduction in 2008, bitcoin has gained much popularity and is currently the most successful digital currency in the world with billions of dollars invested in it. It is built on decades of research in the field of cryptography, digital cash, and distributed computing. In the following section, a brief history is presented in order to provide the background required to understand the foundations behind the invention of bitcoin.

Bitcoin: A Peer-to-Peer Electronic Cash System was written by Satoshi Nakamoto. The first key idea introduced was that purely peer-to-peer electronic cash that does not need an intermediary bank to transfer payments between peers.

Bitcoin is built on decades of cryptographic research such as the research in Merkle trees, hash functions, public key cryptography, and digital signatures. Moreover, ideas such as BitGold, b-money, hashcash, and cryptographic time stamping provided the foundations for bitcoin invention. All these technologies are cleverly combined in bitcoin to create the world's first decentralized currency.

Bitcoin can be defined in various ways: it's a protocol, a digital currency, and a platform. It is a combination of peer-to-peer network, protocols, and software that facilitate the creation and usage of the digital currency named bitcoin. Note that Bitcoin with a capital B is used to refer to the Bitcoin protocol, whereas bitcoin with a lowercase b is used to refer to bitcoin, the currency. Nodes in this peer-to-peer network talk to each other using the Bitcoin protocol.

Decentralization of currency was made possible for the first time with the invention of bitcoin. Moreover, the

double spending problem was solved in an elegant and ingenious way in bitcoin. Double spending problem arises when, for example, a user sends coins to two different users at the same time and they are verified independently as valid transactions.

Bitcoin Working Mechanism:

When you send an email to another person, you just type an email address and can communicate directly to that person. It is the same thing when you send an instant message. This type of communication between two parties is commonly known as Peer-to-Peer communication.

Whenever you want to transfer money to someone over the internet, you need to use a service of third-party such as banks, a credit card, a PayPal, or some other type of money transfer services. The reason for using third-party is to ensure that you are transferring that money. In other words, you need to be able to verify that both parties have done what they need to do in real exchange.

For example, Suppose you click on a photo that you want to send it to another person, so you can simply attach that photo to an email, type the receiver email address and send it. The other person will receive the photo, and you think it would end, but it is not. Now, we have two copies of photo, one is a simple email, and another is an original file which is still on my computer. Here, we send the copy of the file of the photo, not the original file. This issue is commonly known as the double-spend problem.



The double-spend problem provides a challenge to determine whether a transaction is real or not. How you can send a bitcoin to someone over the internet without needing a bank or some other institution to certify the transfer took place. The answer arises in a global network of thousands of computers called a Bitcoin Network and a special type of decentralized ledger technology called **blockchain**.

Transactions & Structure:

Transactions are at the core of the bitcoin ecosystem. Transactions can be as simple as just sending some bitcoins to a bitcoin address, or it can be quite complex depending on the requirements. Each transaction is composed of at least one input and output.

Inputs can be thought of as coins being spent that have been created in a previous transaction and outputs as coins being created. If a transaction is minting new coins, then there is no input and therefore no signature is needed. If a transaction is to send coins to some other user (a bitcoin address), then it needs to be signed by the sender with their private key and a reference is also required to the previous transaction in order to show the origin of the coins. Coins are, in fact, unspent transaction outputs represented in Satoshi.

Transactions are not encrypted and are publicly visible in the blockchain. Blocks are made up of transactions and these can be viewed using any online blockchain explorer.

The transaction life cycle

1. A user/sender sends a transaction using wallet software or some other interface.
2. The wallet software signs the transaction using the sender's private key.
3. The transaction is broadcasted to the Bitcoin network using a flooding algorithm.
4. Mining nodes include this transaction in the next block to be mined.
5. Mining starts once a miner who solves the Proof of Work problem broadcasts the newly mined block to the network.
6. The nodes verify the block and propagate the block further, and confirmation starts to generate.
7. Finally, the confirmations start to appear in the receiver's wallet and after approximately six confirmations, the transaction is considered finalized and confirmed. However, six is just a recommended number, the transaction can be considered final even after the first confirmation. The key idea behind waiting for six confirmations is that the probability of double spending is virtually eliminated after six confirmations.

TRANSACTION STRUCTURE

A transaction at a high level contains metadata, inputs, and outputs. Transactions are combined to create a block. The transaction structure is shown in the following table:

Field	Size	Description
Version Number	4 bytes	Used to specify rules to be used by the miners and nodes for transaction processing.
Input counter	1 bytes – 9 bytes	The number of inputs included in the transaction.
list of inputs	variable	Each input is composed of several fields, including Previous transaction hash, Previous Txout-index, Txin-script length, Txin-script, and optional sequence number. The first transaction in a block is also called a coinbase transaction. It specifies one or more transaction inputs.
Out-counter	1 bytes – 9 bytes	A positive integer representing the number of outputs.
list of outputs	variable	Outputs included in the transaction.
lock_time	4 bytes	This defines the earliest time when a transaction becomes valid. It is either a Unix timestamp or a block number.

MetaData: This part of the transaction contains some values such as the size of the transaction, the number of inputs and outputs, the hash of the transaction, and a lock_time field. Every transaction has a prefix specifying the version number.

Inputs: Generally, each input spends a previous output. Each output is considered an Unspent Transaction Output (UTXO) until an input consumes it.

Outputs: Outputs have only two fields, and they contain instructions for the sending of bitcoins. The first field contains the amount of Satoshis, whereas the second field is a locking script that contains the conditions that need to be met in order for the output to be spent.

Verification: Verification is performed using bitcoin's scripting language.

Types of transaction:

There are various scripts available in bitcoin to handle the value transfer from the source to the destination. These scripts range from very simple to quite complex depending upon the requirements of the transaction. Standard transactions are evaluated using IsStandard() and IsStandardTx() tests and only standard transactions that pass the test are generally allowed to be mined or broadcasted on the bitcoin network. However, nonstandard transactions are valid and allowed on the network.

Pay to Public Key Hash (P2PKH):

P2PKH is the most commonly used transaction type and is used to send transactions to the bitcoin addresses. The format of the transaction is shown as follows:

ScriptPubKey: OP_DUP OP_HASH160 OP_EQUALVERIFY OP_CHECKSIG ScriptSig: The ScriptPubKey and ScriptSig parameters are concatenated together and executed.

Pay to Script Hash (P2SH):

P2SH is used in order to send transactions to a script hash (that is, the addresses starting with 3) and was standardized in BIP16. In addition to passing the script, the redeem script is also evaluated and must be valid. The template is shown as follows:

ScriptPubKey: OP_HASH160 OP_EQUAL

ScriptSig: [...]

MultiSig (Pay to MultiSig): M of n multisignature transaction script is a complex type of script where it is possible to construct a script that required multiple signatures to be valid in order to redeem a transaction. Various complex transactions such as escrow and deposits can be built using this script. The template is shown here:

ScriptPubKey: [. . .] OP_CHECKMULTISIG

ScriptSig: 0 [. . .] Raw multisig is obsolete, and multisig is usually part of the P2SH redeem script, mentioned in the previous bullet point.

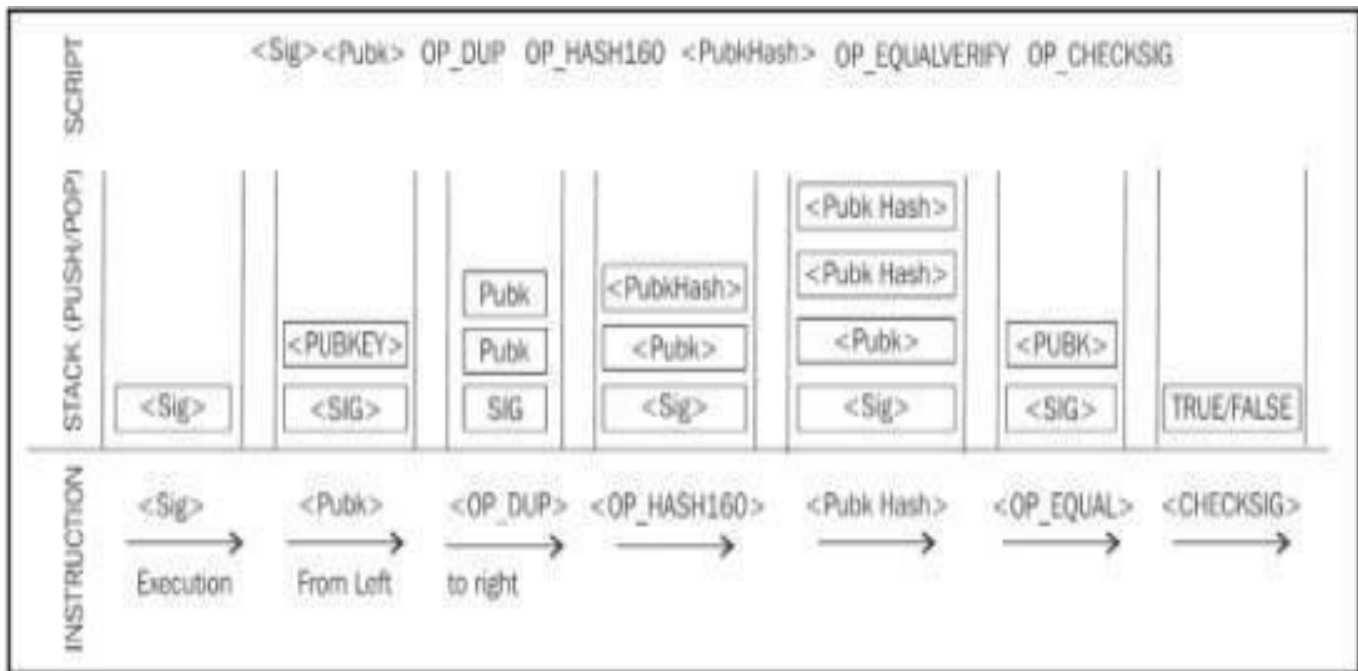
Pay to Pubkey: This script is a very simple script that is commonly used in coinbase transactions. It is now obsolete and was used in an old version of bitcoin. The public key is stored within the script in this case, and the unlocking script is required to sign the transaction with the private key. The template is shown as follows:

OP_CHECKSIG Null data/OP_RETURN: This script is used to store arbitrary data on the blockchain for a fee. The limit of the message is 40 bytes. The output of this script is unredeemable because OP_RETURN will fail the validation in any case. ScriptSig is not required in this case.

The template is very simple and is shown as follows:

OP_RETURN<data>

A P2PKH script execution is shown as follows:



P2PKH script execution:

All transactions are eventually encoded into the hex before transmitting over the bitcoin network.

Blockchain is a public ledger of a timestamped, ordered, and immutable list of all transactions on the bitcoin network. Each block is identified by a hash in the chain and is linked to its previous block by referencing the previous block's hash. In the following structure of a block, a block header is described, followed by a detailed diagram that provides an insight into the blockchain structure.

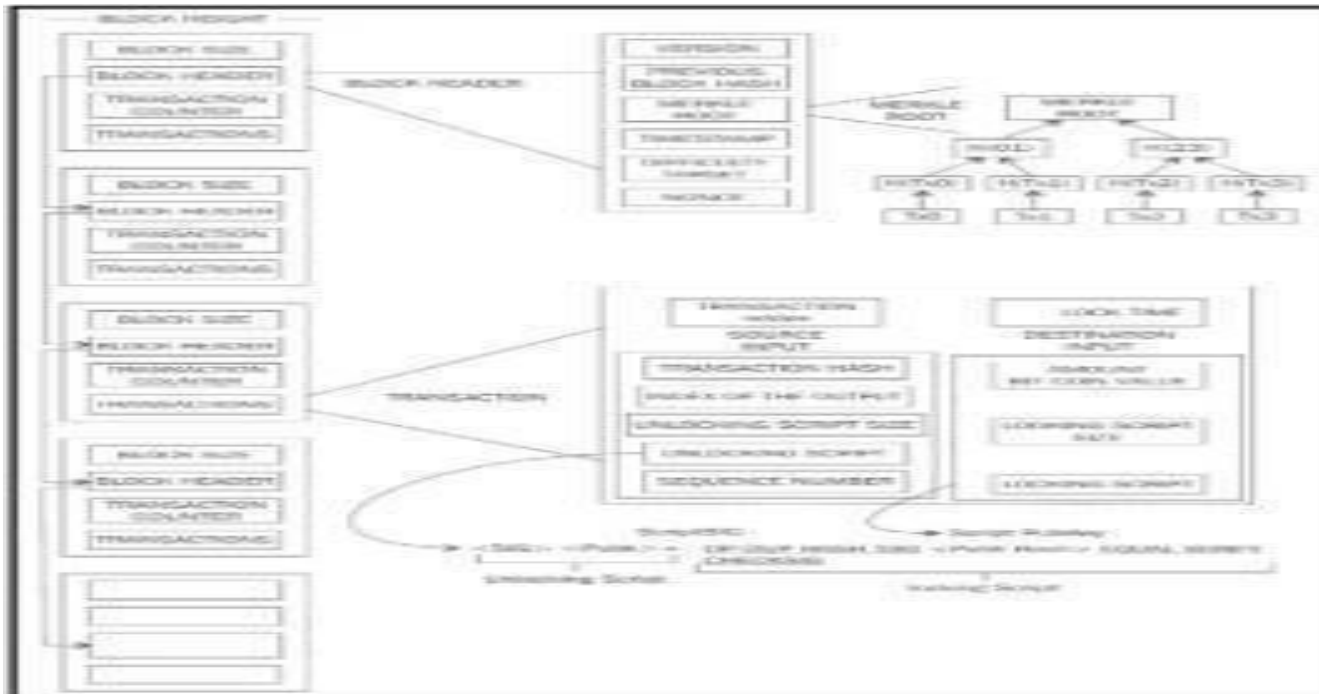
The structure of a block

Bytes	Name	Description
80	Block header	This includes fields from the block header described in the next section.
<i>variable</i>	Transaction counter	The field contains the total number of transactions in the block, including the coinbase transaction.
<i>variable</i>	Transactions	All transactions in the block.

The structure of a block header

Bytes	Name	Description
4	Version	The block version number that dictates the block validation rules to follow.
32	previous block header hash	This is a double SHA256 hash of the previous block's header.
32	merkle root hash	This is a double SHA256 hash of the merkle tree of all transactions included in the block.

4	Timestamp	This field contains the approximate creation time of the block in the Unix epoch time format. More precisely, this is the time when the miner has started hashing the header (the time from the miner's point of view).
4	Difficulty target	This is the difficulty target of the block.
4	Nonce	This is an arbitrary number that miners change repeatedly in order to produce a hash that fulfills the difficulty target threshold.



A visualization of blockchain, block, block header, transaction and script.

As shown in the preceding diagram, blockchain is a chain of blocks where each block is linked to its previous block by referencing the previous block header's hash. This linking makes sure that no transaction can be modified unless the block that records it and all blocks that follow it are also modified. The first block is not linked to any previous block and is known as the genesis block.

The Genesis Block:

This is the first block in the bitcoin blockchain. The genesis block was hardcoded in the bitcoin core software.

Bitcoin provides protection against double spending by enforcing strict rules on transaction verification and via mining. Blocks are added in the blockchain only after strict rule checking and successful Proof of Work solution. Block height is the number of blocks before a particular block in the blockchain. The current height (at the time of writing this) of the blockchain is 434755 blocks. Proof of Work is used to secure the blockchain.

Each block contains one or more transactions, out of which the first transaction is a coinbase transaction. There is a special condition for coinbase transactions that prevent them to be spent until at least 100 blocks in order to avoid a situation where the block may be declared stale later on.

Stale blocks are created when a block is solved and every other miner who is still working to find a solution to the hash puzzle is working on that block. Mining and hash puzzles will be discussed later in the chapter in detail. As the block is no longer required to be worked on, this is considered a stale block.

Orphan Blocks : are also called detached blocks and were accepted at one point in time by the network as valid blocks but were rejected when a proven longer chain was created that did not include this initially accepted block. They are not part of the main chain and can occur at times when two miners manage to produce the blocks at the same time.

The Bitcoin Network :

The bitcoin network is a P2P network where nodes exchange transactions and blocks. There are different types of nodes on the network. There are two main types of nodes, full nodes and SPV nodes. Full nodes, as the name implies, are implementations of bitcoin core clients performing the wallet, miner, full blockchain storage, and network routing functions. However, it is not necessary to perform all these functions. SPV nodes or lightweight clients perform only wallet and network routing functionality. The latest version of Bitcoin protocol is 70014 and was introduced with bitcoin core client 0.13.0.

Bitcoin network is identified by its different magic values. A list is shown as follows:

Network	Magic value	Hex
main	0xD9B4BEF9	F9 BE B4 D9
testnet3	0x0709110B	0B 11 09 07

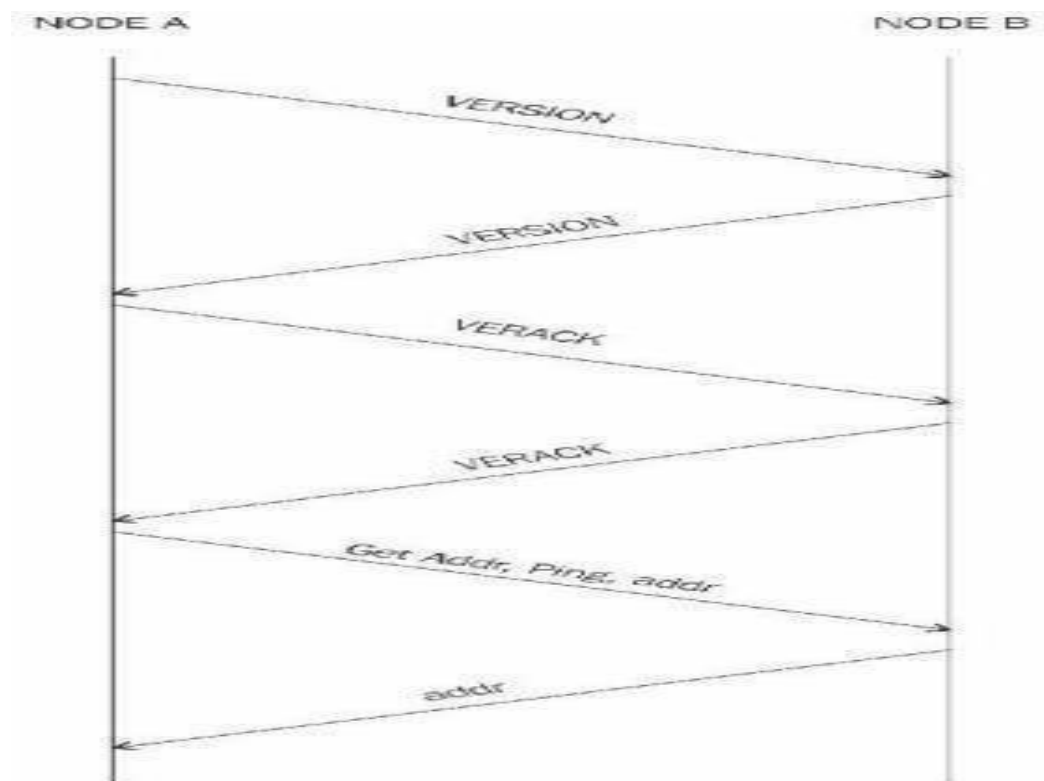
A full node performs four functions: wallet, miner, blockchain, and the network routing node.

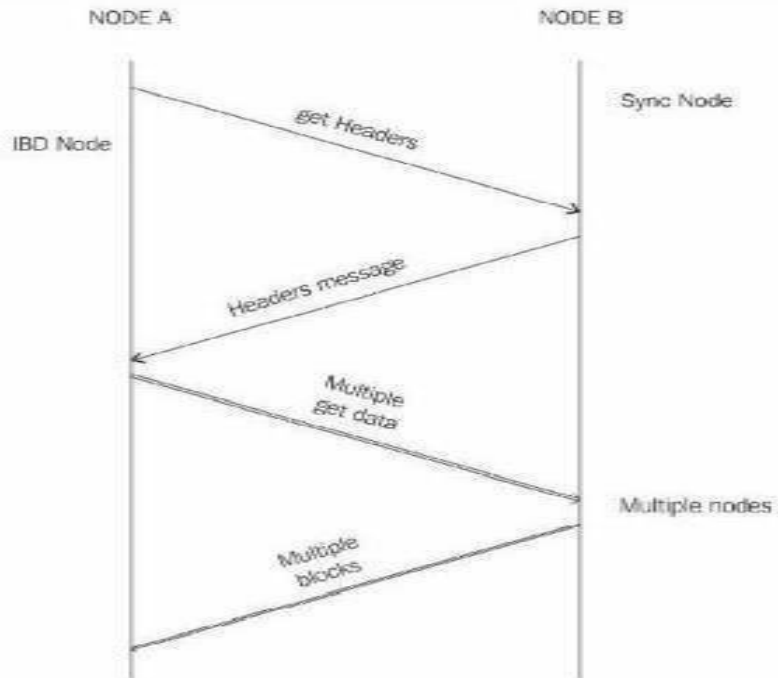
When a bitcoin core node starts up, first, it initiates the discovery of all peers. This is achieved by querying DNS seeds that are hardcoded into the bitcoin core client and are maintained by bitcoin community members. This lookup returns a number of DNS A records. The bitcoin protocol works on TCP port 8333 by default for the main network and TCP 18333 for testnet.

First, the client sends a protocol message Version that contains various fields, such as version, services, timestamp, network address, nonce, and some other fields. The remote node responds with its own version message followed by verack message exchange between both nodes, indicating that the connection has been established.

After this, Getaddr and addr messages are exchanged to find the peers that the client do not know. Meanwhile, either of the nodes can send a ping message to see whether the connection is still live. Now the block download can begin.

If the node already has all blocks fully synchronized, then it listens for new blocks using the Inv protocol message; otherwise, it first checks whether it has a response to inv messages and have inventories already. If yes, then it requests the blocks using the Getdata protocol message; if not, then it requests inventories using the GetBlocks message. This method was used until version 0.9.3





Wallets: The wallet software is used to store private or public keys and bitcoin address. It performs various functions, such as receiving and sending bitcoins. Nowadays, software usually offers both functionalities: bitcoin client and wallet. On the disk, the bitcoin core client wallets are stored as the Berkeley DB file:

```
~/bitcoin$ file wallet.dat
```

wallet.dat: Berkeley DB (Btree, version 9, native byte-order) Private keys can be generated in different ways and are used by different types of wallets.

Wallets do not store any coins, and there is no concept of wallets storing balance or coins for a user. In fact, in the bitcoin network, coins do not exist; instead, only transaction information is stored on the blockchain (more precisely, UTXO unspent outputs), which are then used to calculate the amount of bitcoins.

WALLET TYPES In bitcoin, there are different types of wallets that can be used to store private keys. As a software program, they also provide some functions to the users to manage and carry out transactions on the bitcoin network.

Non-deterministic wallets :

These wallets contain randomly generated private keys and are also called Just a Bunch of Key wallets. The

bitcoin core client generates some keys when first started and generates keys as and when required. Managing a large number of keys is very difficult and an error-prone process can lead to theft and loss of coins. Moreover, there is a need to create regular backups of the keys and protect them appropriately in order to prevent theft or loss.

Deterministic wallets:

In this type of wallet, keys are derived out of a seed value via hash functions. This seed number is generated randomly and is commonly represented by humanreadable mnemonic code words. Mnemonic code words are defined in BIP39. This phrase can be used to recover all keys and makes private key management comparatively easier.

Hierarchical deterministic wallets :

Defined in BIP32 and BIP44, HD wallets store keys in a tree structure derived from a seed. The seed generates the parent key (master key), which is used to generate child keys and, subsequently, grandchild keys. Key generation in HD wallets does not generate keys directly; instead, it produces some information (private key generation information) that can be used to generate a sequence of private keys. The complete hierarchy of private keys in an HD wallet is easily recoverable if the master private key is known. It is because of this property that HD wallets are very easy to maintain and are highly portable.

Brain wallets:

The master private key can also be derived from the hash of passwords that are memorized. The key idea is that this passphrase is used to derive the private key and if used in HD wallets, this can result in a full HD wallet that is derived from a single memorized password. This is known as brain wallet. This method is prone to password guessing and brute force attacks but techniques such as key stretching can be used to slow down the progress made by the attacker. Paper wallets As the name implies, this is a paper-based wallet with the required key material printed on it. It requires physical security to be stored. Paper wallets can be generated online from various service providers, such as <https://bitcoinpaperwallet.com/> or <https://www.bitaddress.org/>.

Hardware wallets:

Another method is to use a tamper-resistant device to store keys. This tamper-resistant device can be custombuilt or with the advent of NFC-enabled phones, this can also be a secure element (SE) in NFC phones. Trezor and Ledger wallets (various types) are the most commonly used bitcoin hardware wallets.



Online wallets :

Online wallets, as the name implies, are stored entirely online and are provided as a service usually via cloud. They provide a web interface to the users to manage their wallets and perform various functions such as making and receiving payments. They are easy to use but imply that the user trusts the online wallet service provider.

Mobile wallets :

Mobile wallets, as the name suggests, are installed on mobile devices. They can provide various methods to make payments, most notably the ability to use smart phone cameras to scan QR codes quickly and make payments. Mobile wallets are available for the Android platform and iOS, for example, breadwallet, copay, and Jaxx.



Jaxx Mobile wallet

Bitcoin payments:

Bitcoins can be accepted as payments using various techniques. Bitcoin is not recognized as a legal currency in many jurisdictions, but it is increasingly being accepted as a payment method by many online merchants and e-commerce websites. There are a numbers of ways in which buyers can pay the business that accepts bitcoins. For example, in an online shop, bitcoin merchant solutions can be used, whereas in traditional physical shops, point of sale terminals and other specialized hardware can be used.

Customers can simply scan the QR barcode with the seller's payment URI in it and pay using their mobile devices. Bitcoin URIs allow users to make payments by simply clicking on links or scanning QR codes. URI (Uniform Resource Identifier) is basically a string that represents the transaction information. It is defined in BIP21.

The QR code can be displayed near the point of the sale terminal. Nearly all bitcoin wallets support this feature. Business can use the following screenshot to advertise that they can accept bitcoins as payment.



Various payment solutions, such as xbtterminal and 34 bytes bitcoin POS terminal are available commercially. 34 bytes POS solution.

Bitcoin payment processor, offered by many online service providers, allows integration with e-commerce websites.

Bitcoin investment and buying and selling bitcoins

There are many online exchanges where users can buy and sell bitcoins. This is a big business on the Internet now and it offers bitcoin trading, CFDs, spread betting, margin trading, and various other choices. Traders can buy bitcoins or trade by opening long or short positions to make profit when bitcoin's price goes up or down. Several other features, such as exchanging bitcoins for other virtual currencies, are also possible, and many

online bitcoin exchanges provide this function. Advanced market data, trading strategies, charts, and relevant data to support traders is also available. An example is shown from CEX.IO here.



Bitcoin installation

The bitcoin core client can be installed from <https://bitcoin.org/en/download>. This is available for different architectures and platforms ranging from x86 windows to ARM Linux, as shown in the following image:



SETTING UP A BITCOIN NODE

A sample run of the bitcoin core installation on Ubuntu is shown here; for other platforms, you can get details from

www.bitcoin.org.

```

drequinox@drequinox-OP7010:~$ sudo apt-add-repository ppa:bitcoin/bitcoin
[sudo] password for drequinox:
Stable channel of bitcoin-qt and bitcoind for Ubuntu, and their dependencies
More info: https://launchpad.net/~bitcoin/+archive/ubuntu/bitcoin
Press [ENTER] to continue or ctrl c to cancel adding it

gpg: keyring '/tmp/tmpas41trw/soaring.gpg' created
gpg: keyring '/tmp/tmpas41trw/pubring.gpg' created
gpg: requesting key 0042C05E from hkp server keyserver.ubuntu.com
gpg: /tmp/tmpas41trw/trustdb.gpg: trustdb created
gpg: key 0042C05E: public key "Launchpad PPA for Bitcoin" imported
gpg: no ultimately trusted keys found
gpg: Total number processed: 1
gpg:       imported: 1 (RSA: 1)
OK
drequinox@drequinox-OP7010:~$

```

Step 2:

```
drequinox@drequinox-OP7010:~$ sudo apt-get update
```

Depending on the client required, users can use either of the following commands, or they can issue both commands at once:

```
sudo apt-get install bitcoind
```

```
sudo apt-get install bitcoin-qt
```

```
drequinox@drequinox-OP7010:~$ sudo apt-get install bitcoin-qt bitcoind
```

Reading package lists... Done Building dependency tree

Reading state information... Done

SETTING UP THE SOURCE CODE

The bitcoin source code can be downloaded and compiled if users wish to participate in the bitcoin code or for

learning purpose. Git can be used to download the bitcoin source code:

```
$ sudo apt-get install git
$ mkdir bcsource
$ cd bcsource
drequinox@drequinox-OP7010:~/bcsource $ git clone https://github.com/bitcoin/bitcoin.git
Cloning into 'bitcoin'...
remote:
Counting objects: 78960, done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 78960 (delta 0), reused 0 (delta 0), pack-reused 78957
Receiving objects: 100% (78960/78960), 72.53 MiB | 1.85 MiB/s, done.
Resolving deltas: 100% (57908/57908), done.
Checking connectivity... done.
```

Change the directory to bitcoin:

```
drequinox@drequinox-OP7010:~/bcsource$ cd bitcoin
```

After the preceding steps are completed, the code can be compiled:

```
drequinox@drequinoxOP7010:~/bcsource/bitcoin$ ./autogen.sh
drequinox@drequinoxOP7010:~/bcsource/bitcoin$ ./configure.sh
drequinox@drequinoxOP7010:~/bcsource/bitcoin$ make drequinox@drequinoxOP7010:~/bcsource/bitcoin$
sudo make install
```

SETTING UP BITCOIN.CONF

bitcoin.conf file is a configuration file that is used by the bitcoin core client to save configuration settings. All command line options for the bitcoind client with the exception of -conf switch can be set up in the configuration file, and when bitcoin-qt or bitcoind will start up, it will take the configuration information from that file. In Linux systems, this is usually found in \$HOME/.bitcoin/, or it can also specified in the command line using the -conf= switch to bitcoind core client software.

STARTING UP A NODE IN TESTNET

The bitcoin node can be started in the testnet mode if you want to test the bitcoin network and run an experiment. This is a faster network as compared to the live network and has relaxed rules for mining and transactions. Various faucet services are available for the bitcoin test network. One example is Bitcoin TestNet sandbox, where users can request bitcoins to be paid to their testnet bitcoin address. This can be accessed via <https://testnet.manu.backend.hamburg/>. This is very useful for experimentation with transactions on test net.

The command line to start up test net is as follows:

```
bitcoind --testnet -daemon
```

```
bitcoin-cli --testnet
```

```
bitcoin-qt --testnet
```

STARTING UP A NODE IN REGTEST

The regtest mode (regression testing mode) can be used to create a local blockchain for testing purposes. The following commands can be used to start up a node in the reg test mode

```
bitcoind -regtest -daemon
```

Bitcoin server starting

Blocks can be generated using the following command:

```
bitcoin-cli -regtest
```

generate 200 Relevant log messages can be viewed in the .bitcoin/regtest directory on a Linux system under debug.log.

After block generation, the balance can be viewed as follows:

```
drequinox@drequinoxOP7010:
```

```
~/bitcoin/regtest
```

```
$ bitcoin-cli -regtest getbalance 8750.00000000
```

The node can be stopped using this:

```
drequinox@drequinox-OP7010:~/bitcoin$ bitcoin-cli -regtest stop
```

Bitcoin server stopping

STARTING UP A NODE IN LIVE MAINNET

Bitcoin-d is the core client software that can be run as a daemon, and it provides the JSON RPC interface. Bitcoin-cli is the command line feature-rich tool to interact with the daemon; the daemon then interacts with the blockchain and performs various functions. Bitcoin-cli calls only JSON-RPC functions and does not perform any actions on its own on the blockchain.

Bitcoin-qt is the bitcoin core client GUI. When the wallet software starts up first, it verifies the blocks on the disk and then starts up and shows the following GUI:

Bitcoin Core QT client, just after installation, showing that blockchain is not in sync. The verification process is not specific to the Bitcoin-qt client; it is performed by the bitcoind client as well. **EXPERIMENTING WITH BITCOIN CLI**

Bitcoin-cli is the command-line interface available with the bitcoin core client and can be used to perform various functions using the RPC interface provided by the bitcoin core client. A sample run of bitcoin-cli getinfo; the same format can be used to invoke other commands. A list of all commands can be shown via the following command: Testnet bitcoin-cli, this is just the first few lines of the output, actual output has many commands.

HTTP REST: Starting from bitcoin core client 0.10.0, the HTTP REST interface is also available. By default, this runs on the same TCP port 8332 as JSON-RPC.

Bitcoin programming and the command-line interface

Bitcoin programming is a very rich field now. The bitcoin core client exposes various JSON RPC commands that can be used to construct raw transactions and perform other functions via custom scripts or programs. Also, the command line tool Bitcoin-cli is available, which makes use of the JSON-RPC interface and provides a rich toolset to work with Bitcoin.

These APIs are also available via many online service providers in the form of bitcoin APIs, and they provide a simple HTTP REST interface. Bitcoin APIs, such as blockchain.info and bitpay, block.io, and many others, offer a myriad of options to develop bitcoin-based solutions. Various libraries are available for bitcoin programming. A list is shown as follows, and those if you are interested can further explore the libraries.

Libbitcoin: Available at <https://libbitcoin.dyne.org/> and provides powerful command line utilities and clients.

Pycoin: Available at <https://github.com/richardkiss/pycoin>, is a library for Python.

Bitcoinj: This library is available at <https://bitcoinj.github.io/> and is implemented in Java.

There are many online bitcoin APIs available, the most commonly used APIs are listed as follows:

<https://bitcore.io/>

<https://bitcoinjs.org/>

<https://blockchain.info/api>

All APIs offer more or less the same type of functionality, and it gets difficult to choose which API is the best.

Bitcoin improvement proposals (BIPs) :

These documents are used to propose or inform the bitcoin community about the improvements suggested, the design issues, or information about some aspects of the bitcoin ecosystem. There are three types of bitcoin improvement proposals, abbreviated as BIPs:

Standard BIP: Used to describe the major changes that have a major impact on the bitcoin system, for example, block size changes, network protocol changes, or transaction verification changes.

Process BIP: A major difference between standard and process BIPs is that standard BIPs cover protocol changes, whereas process BIPs usually deal with proposing a change in a process that is outside the core Bitcoin protocol. These are implemented only after a consensus among bitcoin users.

Informational BIP: These are usually used to just advise or record some information about the bitcoin ecosystem, such as design issues.

UNIT-4

ETHEREUM :

Ethereum is a platform powered by blockchain technology that is best known for its native cryptocurrency, called ether, or ETH, or simply ethereum. The distributed nature of blockchain technology is what makes the Ethereum platform secure, and that security enables ETH to accrue value.

Ethereum, just like any other blockchain, can be visualized as a transaction-based state machine. The idea is that a genesis state is transformed into a final state by executing transactions incrementally. The final transformation is then accepted as the absolute undisputed version of the state. In the following diagram, the Ethereum state transition function is shown, where a transaction execution has resulted in a state transition.

Elements of Ethereum Block chain:

In the following section, you will be introduced to various components of the Ethereum network and the blockchain. First, the basic concept of the EVM is given in the next section.

Ethereum virtual machine(EVM)

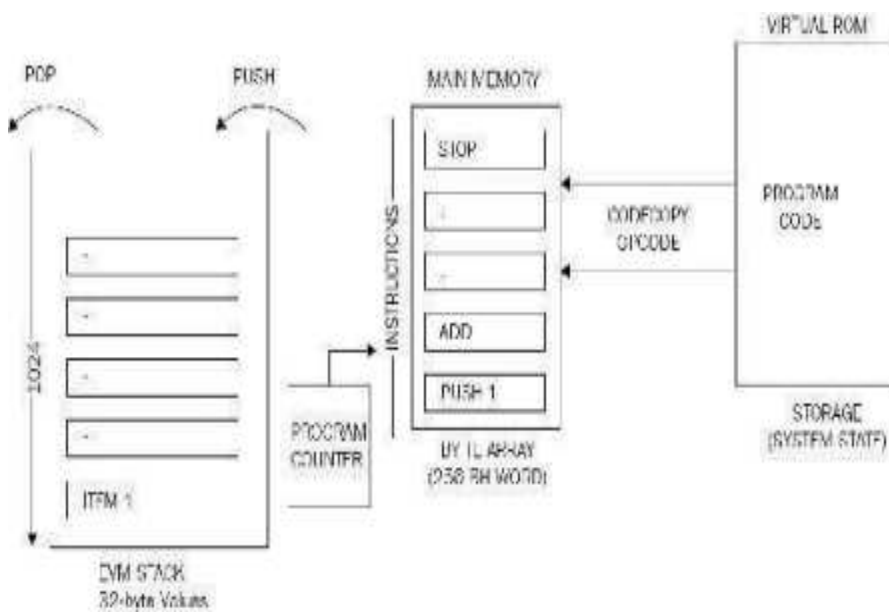
EVM is a simple stack-based execution machine that runs bytecode instructions in order to transform the system state from one state to another. The word size of the virtual machine is set to 256-bit. The stack size is limited to 1024 elements and is based on the **LIFO (Last in First Out)** queue. EVM is a Turing-complete machine but is limited by the amount of gas that is required to run any instruction. This means that infinite loops that can result in denial of service attacks are not possible due to gas requirements.

EVM also supports exception handling in case exceptions occur, such as not having enough gas or invalid instructions, in which case the machine would immediately halt and return the error to the executing agent. EVM is a fully isolated and sandboxed runtime environment. EVM is a stack-based architecture. EVM is big-endian by design and it uses 256-bit wide words. This word size allows for Keccak 256-bit hash and elliptic curve cryptography computations.

EVM also supports exception handling in case exceptions occur, such as not having enough gas or invalid instructions, in which case the machine would immediately halt and return the error to the executing agent. EVM is a fully isolated and sandboxed runtime environment.

As discussed earlier, EVM is a stack-based architecture. EVM is big-endian by design and it uses 256-bit wide words. This word size allows for Keccak 256-bit hash and elliptic curve cryptography computations.

The following diagram shows the design of the EVM where the virtual ROM stores the program code that is copied into main memory using CODECOPY. The main memory is then read by the EVM by referring to the program counter and executes instructions



EVM operation

EVM optimization is an active area of research and recent research has suggested that EVM can be optimized and tuned to a very fine degree in order to achieve high performance. Research into the possibility of using Web assembly (WASM) is underway already. WASM is developed by Google, Mozilla, and Microsoft and is now being designed as an open standard by the W3C community group. The aim of WASM is to be able to run machine code in the browser that will result in execution at native speed. Similarly, the aim of EVM 2.0 is to be able to run the EVM instruction set (Opcodes) natively in CPUs, thus making it faster and efficient.

PRE-COMPILED CONTRACTS:

There are four precompiled contracts in Ethereum. Here is the list of these contracts and details.

The elliptic curve public key recovery function

ECDSARECOVER (Elliptic curve DSA recover function) is available at address 1. It is denoted as ECREC and requires 3000 gas for execution. If the signature is invalid, then no output is returned by this function. Public key recovery is a standard mechanism by which the public key can be derived from the private key in elliptic curve cryptography.

The ECDSA recovery function is shown as follows:

ECDSARECOVER(H, V, R, S) = Public Key

It takes four inputs: H, which is a 32 byte hash of the message to be signed and V, R, and S, which represent the ECDSA signature with the recovery ID and produce a 64 byte public key. V, R, and S have been discussed in detail previously in this chapter.

The SHA-256 bit hash function

The SHA-256 bit hash function is a precompiled

contract that is available at address 2 and produces a SHA256 hash of the input. It is almost like a pass-through function. Gas requirement for SHA-256 (SHA256) depends on the input data size. The output is a 32 byte value.

The RIPEMD-160 bit hash function

The RIPEMD-160 bit hash function is used to provide RIPEMD 160-bit hash and is available at address 3. The output of this function is a 20-byte value. Gas requirement, similar to SHA-256, is dependent on the amount of input data.

The identity function:

The identity function is available at address 4 and is denoted by the ID. It simply defines output as input; in

other words, whatever input is given to the ID function, it will output the same value. Gas requirement is calculated by a simple formula: $15 + 3 \lceil I_d / 32 \rceil$ where I_d is the input data. This means that at a high level, the gas requirement is dependent on the size of the input data albeit with some calculation performed, as shown in the preceding equation. All the previously mentioned precompiled contracts can become native extensions and can be included in the EVM opcodes in the future.

ACCOUNTS AND ITS TYPES :

Accounts are one of the main building blocks of the Ethereum blockchain. The state is created or updated as a result of the interaction between accounts. Operations performed between and on the accounts represent state transitions. State transition is achieved using what's called the Ethereum state transition function, which works as follows:

1. Confirm the transaction validity by checking the syntax, signature validity, and nonce.
2. Transaction fee is calculated and the sending address is resolved using the signature. Furthermore, sender's account balance is checked and subtracted accordingly and nonce is incremented. An error is returned if the account balance is not enough.
3. Provide enough ether (gas price) to cover the cost of the transaction. This is charged per byte incrementally according to the size of the transaction.
4. In this step, the actual transfer of value occurs. The flow is from the sender's account to receiver's account. The account is created automatically if the destination account specified in the transaction does not exist yet. If the destination account is a contract, then the contract code is executed. If enough gas is available, then the contract code will be executed fully; otherwise, it will run up to the point where it runs out of gas.
5. In cases of transaction failure due to insufficient account balance or gas, all state changes are rolled back with the exception of fee payment, which is paid to the miners.
6. Finally, the remainder (if any) of the fee is sent back to the sender as change and fee is paid to the miners accordingly. At this point, the function returns the resulting state.

TYPES OF ACCOUNTS : There are two types of accounts in Ethereum:

- Externally owned contacts
- Contract accounts

The first is **externally owned accounts (EOAs)** and the other is contract accounts. EOAs are similar to accounts that are controlled by a private key in bitcoin. Contract accounts are the accounts that have code associated with them along with the private key. An EOA has ether balance, is able to send transactions, and has no associated code, whereas a **Contract Account (CA)** has ether balance, associated code, and the ability to get triggered and execute code in response to a transaction or a message that due to the Turing-completeness property of the Ethereum blockchain, the code within contract accounts can be of any level of complexity. The code is executed by EVM by each mining node on the Ethereum network. In addition, contract accounts are able to maintain their own permanent state and can call other contracts. It is envisaged that in the serenity release, the distinction between externally owned accounts and contract accounts may be eliminated.

Block:

As discussed earlier, blocks are the main building blocks of a blockchain. Ethereum blocks consist of various components, which are described as follows:

- The block header
- The transactions list
- The list of headers of ommers or uncles

The transaction list is simply a list of all transactions included in the block. In addition, the list of headers of Uncles is also included in the block. The most important and complex part is the block header.

BLOCK HEADER

Block headers are the most critical and detailed components of an Ethereum block. The header contains valuable information, which is described in detail here.

PARENT HASH

This is the Keccak 256-bit hash of the parent (previous) block's header.

OMMERS HASH

This is the Keccak 256-bit hash of the list of Ommers (Uncles) blocks included in the block.

BENEFICIARY

Beneficiary field contains the 160-bit address of the recipient that will receive the mining reward once the block is successfully mined.

STATE ROOT

The state root field contains the Keccak 256-bit hash of the root node of the state trie. It is calculated after all transactions have been processed and finalized.

TRANSACTIONS ROOT

The transaction root is the Keccak 256-bit hash of the root node of the transaction trie. Transaction trie represents the list of transactions included in the block.

RECEIPTS ROOT

The receipts root is the keccak 256 bit hash of the root node of the transaction receipt trie. This trie is composed of receipts of all transactions included in the block.

Transaction receipts are generated after each transaction is processed and contain useful post-transaction information. More details on transaction receipts.

LOGS BLOOM

The logs bloom is a bloom filter that is composed of the logger address and log topics from the log entry of each transaction receipt of the included transaction list in the block. Logging is explained in detail in the next section.

DIFFICULTY

The difficulty level of the current block.

NUMBER

The total number of all previous blocks; the genesis block is block zero.

GAS LIMIT: The field contains the value that represents the limit set on the gas consumption per block.

GAS USED: The field contains the total gas consumed by the transactions included in the block.

TIMESTAMP : Timestamp is the epoch Unix time of the time of block initialization.

EXTRA DATA: Extra data field can be used to store arbitrary data related to the block.

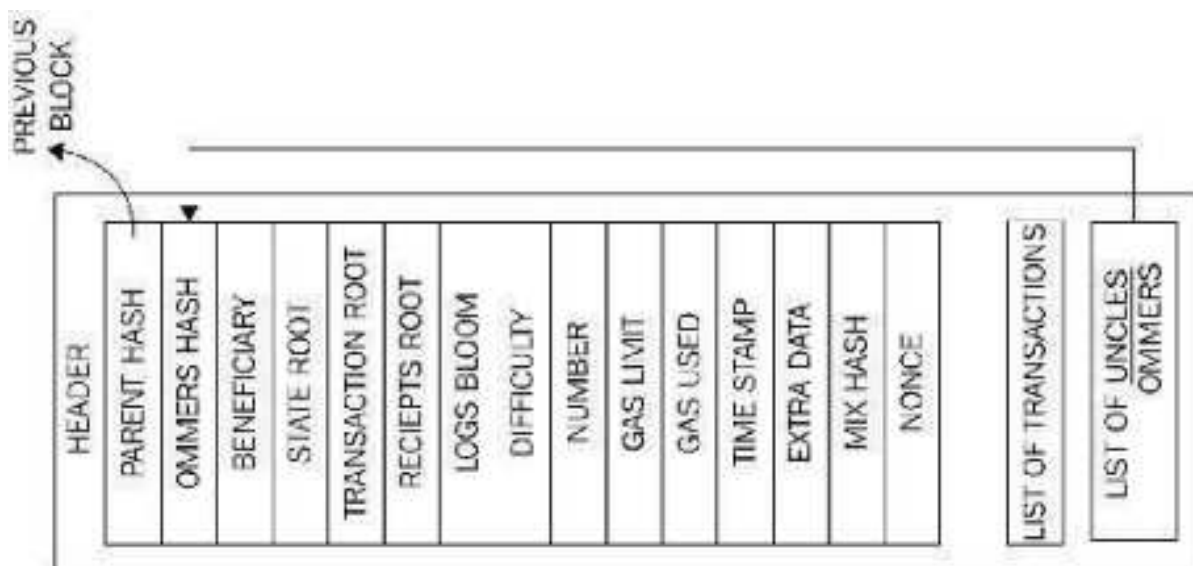
MIXHASH:

Mixhash field contains a 256-bit hash that once combined with the nonce is used to prove that adequate computational effort has been spent in order to create this block.

NONCE

Nonce is a 64-bit hash (a number) that is used to prove, in combination with the mixhash field, that adequate computational effort has been spent in order to create this block.

The following figure shows the detailed structure of the block and block header:



Detailed diagram of block structure with block header

ETHER:

Ether is minted by miners as a currency reward for the computational effort they spend in order to secure the network by verifying and with validation transactions and blocks. Ether is used within the Ethereum blockchain to pay for the execution of contracts on the EVM. Ether is used to purchase gas as crypto fuel, which is required in order to perform computation on the Ethereum blockchain.

The denomination table is shown as follows:

Unit	Wei Value	Weis
Wei	1 Wei	1
Babbage	1e3 Wei	1,000
Lovelace	1e6 Wei	1,000,000
Shannon	1e9 Wei	1,000,000,000
Szabo	1e12 Wei	1,000,000,000,000
Finney	1e15 Wei	1,000,000,000,000,000
Ether	1e18 Wei	1,000,000,000,000,000,000

Fees are charged for each computation performed by the EVM on the blockchain.

Gas

Gas is required to be paid for every operation performed on the Ethereum blockchain. This is a mechanism that ensures that infinite loops cannot cause the whole blockchain to stall due to the Turing-complete nature of the EVM. A fee is paid for transactions to be included by miners for mining. If this fee is too low, the transaction may never be picked up; the more the fee, the higher are the chances that the transactions will be picked up by the miners for inclusion in the block.

Conversely, if the transaction that has an appropriate fee paid is included in the block by miners but has too many complex operations to perform, it can result in an out-of-gas exception if the gas cost is not enough. In this case, the transaction will fail but will still be made part of the block and the transaction originator will not get any refund.

Transaction cost can be estimated using the following formula:

$$\text{Total cost} = \text{gasUsed} * \text{gasPrice}$$

Here, gasUsed is the total gas that is supposed to be used by the transaction during the execution. This is specified in Ether. Each EVM opcode has a fee assigned to it. It is an estimate because the gas used can be more or less than the value specified by the transaction originator originally.

- For example, if computation takes too long or the behavior of the smart contract changes in response to some other factors, then the transaction execution may perform more or less operations than originally intended and can result in consuming more or fewer gas.

Each operation costs some gas; a high level fee schedule of a few operations is shown as an example here:

Operation Name	Gas Cost
step	1
stop	0
suicide	0
sha3	30
sload	20
txdata	5
transaction	500
contract creation	53000

Based on the preceding fee schedule and the formula discussed earlier, an example calculation of the SHA3 operation can be calculated as follows:

- SHA3 costs 30 gas
- Current gas price is 25 GWei, which is 0.000000025
- Ether Multiplying both: $0.000000025 * 30 = 0.000000075$

Ether In total, 0.000000075 Ether is the total gas that will be charged.

Fee schedule

Gas is charged in three scenarios as a prerequisite to the execution of an operation:

- The computation of an operation
- For contract creation or message call

- Increase in the usage of memory

A list of instructions and various operations with the gas values has been provided.

MESSAGES:

- Messages, as defined in the yellow paper, are the data and value that are passed between two accounts. A message is a data packet passed between two accounts. This data packet contains data and value (amount of ether).
- Contracts can send messages to other contracts. Messages only exist in the execution environment and are never stored. Messages are similar to transactions; however, the main difference is that they are produced by the contracts, whereas transactions are produced by entities external to the Ethereum environment.

A message consists of the components mentioned here:

- Sender of the message
- Recipient of the message
- Amount of Wei to transfer and message to the contract address
- Optional data field (Input data for the contract)
- Maximum amount of gas that can be consumed

Messages are generated when CALL or DELEGATECALL opcodes are executed by the contracts.

MINING:

Mining is the process by which new currency is added to the blockchain. This is an incentive for the miners to validate and verify blocks made up of transactions. The mining process helps secure the network by verifying computations.

At a theoretical level, a miner performs the following functions:

- Listens for the transactions broadcasted on the Ethereum network and determines the transactions to be processed.
- Determines stale blocks called Uncles or Ommers and includes them in the block.

- Updates the account balance with the reward earned from successfully mining the block.
- Finally, a valid state is computed and block is finalized, which defines the result of all state transition.

The current method of mining is based on Proof of Work, which is similar to that of bitcoin, but it must also contain the Proof of Work for a given difficulty.

Considerable research work has been carried out in order to build the Proof of Stake algorithm suitable for the Ethereum Network.

various methods of mining are mentioned.

CPU mining

Even though not profitable on the main net, CPU mining is still valuable on the test network or even a private network to experiment with mining and contract deployment. Private and test networks will be discussed with practical examples in the next chapter. A geth example is shown on how to start CPU mining here.

Geth can be started with mine switch in order to start mining:

```
geth --mine --minerthreads <n>
```

CPU mining can also be started using the web 3 geth console. Geth console can be started by issuing the following command:

```
geth attach
```

After this, the miner can be started by issuing the following command, which will return true if successful, or false otherwise. Take a look at the following command:

```
Miner.start(4)
True
```

The preceding command will start the miner with four threads. Take a look at the following command:

```
Miner.stop
True
```

The preceding command will stop the miner. The command will return true if successful.

GPU mining

At a basic level, GPU mining can be performed easily by running two commands:

```
geth --rpc
```

Once geth is up and running and the blockchain is fully downloaded, Ethminer can be run in order to start mining. Ethminer is a standalone miner that can also be used in the farm mode to contribute to mining pools. It can be downloaded from <https://github.com/Genoil/cpp-ethereum/tree/master/releases>:

```
ethminer -G
```

Mining rigs

Mining rigs can be built with some effort and are also available commercially from various vendors. A typical mining rig configuration includes the components discussed in the upcoming sections.

MOTHERBOARD

A specialized motherboard with multiple PCI-E x1 or x16 slots, for example, BIOSTAR Hi-Fi or ASRock H81, is required

SSD HARD DRIVE

An SSD hard drive is required. The SSD drive is recommended because of its much faster performance over the analog equivalent. This will be mainly used to store the blockchain.

GPU

The GPU is the most important component of the rig as it is the main workhorse that will be used for mining. For example, it can be a Sapphire AMD Radeon R9 380 with 4 GB RAM

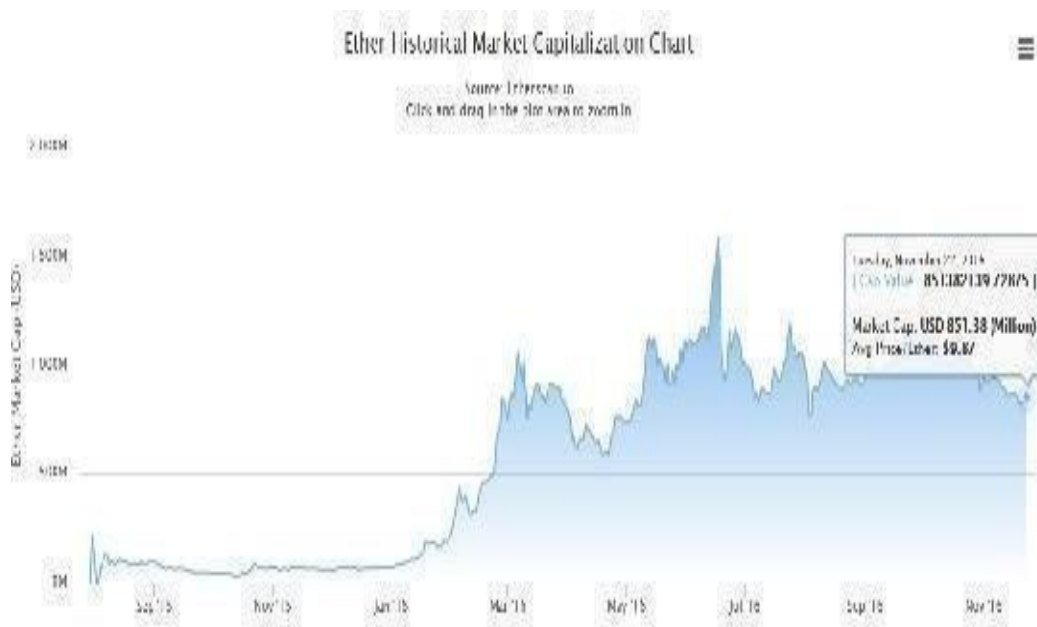
Mining pools

There are many online mining pools that offer Ethereum mining. Ethminer can be used to connect to a mining pool using the following command. Each pool publishes its own instructions, but generally, the process of connecting to a pool is similar.

TRADING AND INVESTMENT:

Ether is available at various exchanges for buying and selling. The current market cap of Ethereum is £680,277,967 at the time of writing this, and an Ether is worth £7.89. Recently, the price has been very volatile and has dropped down significantly due to recent Ethereum attacks and subsequent forks on the Ethereum network.

The following chart shows the historical market capitalization details:



Ether historical market capitalization (source Etherscan.io). There are online services available, such as shapeshift.io, that allow conversion from one currency to another. Various online exchanges, such as Kraken, Coinbase, and many more, offer ether to be purchased for fiat currency using credit cards or another virtual currency, such as bitcoin.

THE YELLOW PAPER

The Ethereum yellow paper has been written by *Dr. Gavin Wood* and serves as a formal definition of the Ethereum protocol. Anyone can implement an Ethereum client by following the protocol specifications defined in the paper. This paper can be somewhat difficult to read, especially for the readers who do not have a background in algebra or mathematics and are not familiar with mathematical notations.

Once symbol meanings are known, it becomes quite easy to understand and appreciate the concepts and specifications described in the yellow paper.

THE ETHEREUM NETWORK:

The Ethereum network is a peer-to-peer network where nodes participate in order to maintain the blockchain and contribute to the consensus mechanism. Networks can be divided into three types, based on requirements and usage.

MainNet : MainNet is the current live network of ethereum. The current version of MainNet is homestead.

TestNet : TestNet is also called Ropsten and is the test network for the Ethereum blockchain. This blockchain is used to test smart contracts and DApps before being deployed to the production live blockchain. Moreover, being a test network, it allows experimentation and research.

Private net(s):

As the name suggests, this is the private network that can be created by generating a new genesis block. This is usually the case in distributed ledger networks, where a private group of entities start their own blockchain.

Supporting protocols:

There are various supporting protocols that are in development in order to support the complete decentralized ecosystem. This includes whisper and Swarm protocol.

Whisper

Whisper provides decentralized peer-to-peer messaging capabilities to the ethereum network. In essence, whisper is a communication protocol that nodes use in order to communicate with each other.

SWARM

Swarm is being developed as a distributed file storage platform. It is a decentralized, distributed, and peer-to-peer storage network. Files in this network are addressed by the hash of their content.

APPLICATIONS DEVELOPED ON ETHEREUM:

There are various implementations of DAOs and smart contracts in Ethereum, most notably, *the DAO*, which was recently hacked and required a hard fork in order for funds to be recovered. The DAO was created to serve as a decentralized platform to collect and distribute investments.

Augur is another DAPP that has been implemented on Ethereum, which is a decentralized prediction market. Various other decentralized applications are listed on <http://dapps.ethercasts.com/>.

SCALABILITY AND SECURITY ISSUES:

Scalability in any blockchain is a fundamental issue. Security is also of paramount importance. Issues such as privacy and confidentiality have caused some adaptability issues, especially in the financial sector. However, a great deal of research is being conducted in these areas. Even though various use cases and proof of concept systems have been developed and the technology works well for many of the scenarios, there still is a need to address some fundamental limitations that are present in blockchains in order to make this technology more adaptable.

At the top of the list of these issues comes scalability and then privacy. Both of these are important limitations to address, especially as blockchains are envisioned to be used in privacy-demanding industries too. There are specific requirements around confidentiality of transactions in finance, law and health, whereas scalability is generally a concern where blockchains do not meet the adequate performance levels expected by the users. These two issues are becoming inhibiting factors toward blockchain technology's wider acceptance.

Scalability:

This is the single most important problem that could mean the difference between wider adaptability of blockchains or limited private use only by consortiums. As a result of substantial research in this area, many solutions have been proposed from a theoretical perspective, the general approach toward tackling the scalability issue generally revolves around protocol-level enhancements. For example, a commonly mentioned solution to bitcoin scalability is to increase its block size. Other proposals include off-chain solutions that offload certain processing to off-chain networks, for example, off-chain state networks. Based on the solutions mentioned

above, generally, the proposals can be divided into two categories: on-chain solutions that are based on the idea of changing fundamental protocols on which the blockchain operates.

Privacy:

Privacy of transactions is a much desired property of blockchains. However, due to its very nature, especially in public blockchains, everything is transparent, thus inhibiting its usage in various industries where privacy is of paramount importance, such as finance, health, and many others. There are different proposals made to address the privacy issue and some progress has already been made. Several techniques, such as indistinguishability obfuscation, usage of homomorphic encryption, zero knowledge proofs, and ring signatures.

Security:

Even though blockchains are generally secure and make use of asymmetric and symmetric cryptography as required throughout the blockchain network, there still are few caveats that can result in compromising the security of the blockchain.

There are a few examples of transaction malleability, eclipse attacks, and possibility of double spending in bitcoin that, in certain scenarios, have been shown to work by various researchers.

UNIT-5

Smart Contract :

Smart contracts are now an ongoing and intense area of research in the blockchain space. Many blockchains have emerged that support smart contracts.

Due to benefits such as the increased security, cost-saving, and transparency that smart contracts can bring to many industries (especially the finance industry), rigorous research is in progress at various commercial and academic institutions to make the implementation of smart contracts easier, more practical, business-friendly, and more secure as soon as possible.

A smart contract is a secure and unstoppable computer program representing an agreement that is automatically executable and enforceable.

Dissecting this definition reveals that a smart contract is, fundamentally, a computer program that is written in a language that a computer or target machine can understand. Also, it encompasses agreements between parties in the form of business logic. Another fundamental idea is that smart contracts are automatically executed according to the instruction that is coded in.

Ricardian contracts:

Ricardian contracts were initially used in a bond trading and payment system called **Ricardo**. The fundamental idea behind this contract is to write a document that is understood and accepted by both a court of law and computer software. Ricardian contracts address the challenge of the issuance of value over the internet. A Ricardian contract identifies the issuer and captures all the terms and clauses of the contract in a document to make it acceptable as a legally binding contract.

A Ricardian contract is a document that has several of the following properties:

- It is a contract offered by an issuer to holders
- It is a valuable right held by holders and managed by the issuer

- It can be easily read by people (like a contract on paper)
- It can be read by programs (parsable, like a database)
- It is digitally signed
- It carries the keys and server information
- It is allied with a unique and secure identifier

Smart contracts can be implemented in any industry where they are required, but the most popular use cases relate to the financial sector. This is because blockchain first found many use cases in the finance industry and, therefore, sparked enormous research interest in the financial industry long before other areas. Recent work in the smart contract space specific to the financial sector has proposed the idea of smart contract templates.

Smart contracts may or may not be deployed on a blockchain, but it makes sense to do so on a blockchain due to the security and decentralized consensus mechanism provided by the blockchain.

The DAO

The **Decentralized Autonomous Organization (DAO)**, started in April 2016, was a smart contract written to provide a platform for investment. Due to a bug, called the **reentrancy bug**, in the code, it was hacked in June 2016. An equivalent of approximately 3.6 million ether (roughly 50 million US dollars) was siphoned out of the DAO into another account.

Even though the term hacked is used here, it was not really hacked. The smart contract did what it was asked to do but due to the vulnerabilities in the smart contracts, the attacker was able to exploit it. It can be seen as an unintentional behavior (a bug) that programmers of the DAO did not foresee. This incident resulted in a hard fork on the Ethereum blockchain, which was introduced to recover from the attack.

The DAO attack exploited a vulnerability (reentrancy bug) in the DAO code where it was possible to withdraw tokens from the DAO smart contract repeatedly before giving the DAO contract a chance to update.

Hyperledger:

Hyperledger is not a blockchain, but a project that was initiated by the Linux Foundation in December 2015 to advance blockchain technology. This project is a collaborative effort by its members to build an open source distributed ledger framework that can be used to develop and implement cross-industry blockchain applications and systems. The principal focus is to create and run platforms that support global business transactions. The project also focuses on improving the reliability and performance of blockchain systems.

Projects under Hyperledger:

There are four categories of projects under Hyperledger. Under each category, there are multiple projects. The categories are:

- Distributed ledgers
- Libraries
- Tools
- Domain-specific

Currently, there are six distributed ledger projects under the

Hyperledger umbrella: **Fabric**, **Sawtooth**, **Iroha**, **Indy**, **Besu**, and **Burrow**. Under libraries, there are the **Aries**, **Transact**, **Quilt**, and **Ursa** projects. The tools category of Hyperledger includes projects such as **Avalon**, **Cello**, **Caliper**, and **Explorer**. There are also domain-specific projects such as Hyperledger **Grid** and Hyperledger **Labs**.

Hyperledger reference architecture:

Hyperledger has published a white paper that presents a reference architecture model that can serve as a guideline to build permissioned distributed ledgers. The reference architecture consists of various components that form a business blockchain.

These high-level components are shown in the reference architecture diagram here, which has been drawn from the aforementioned white paper:

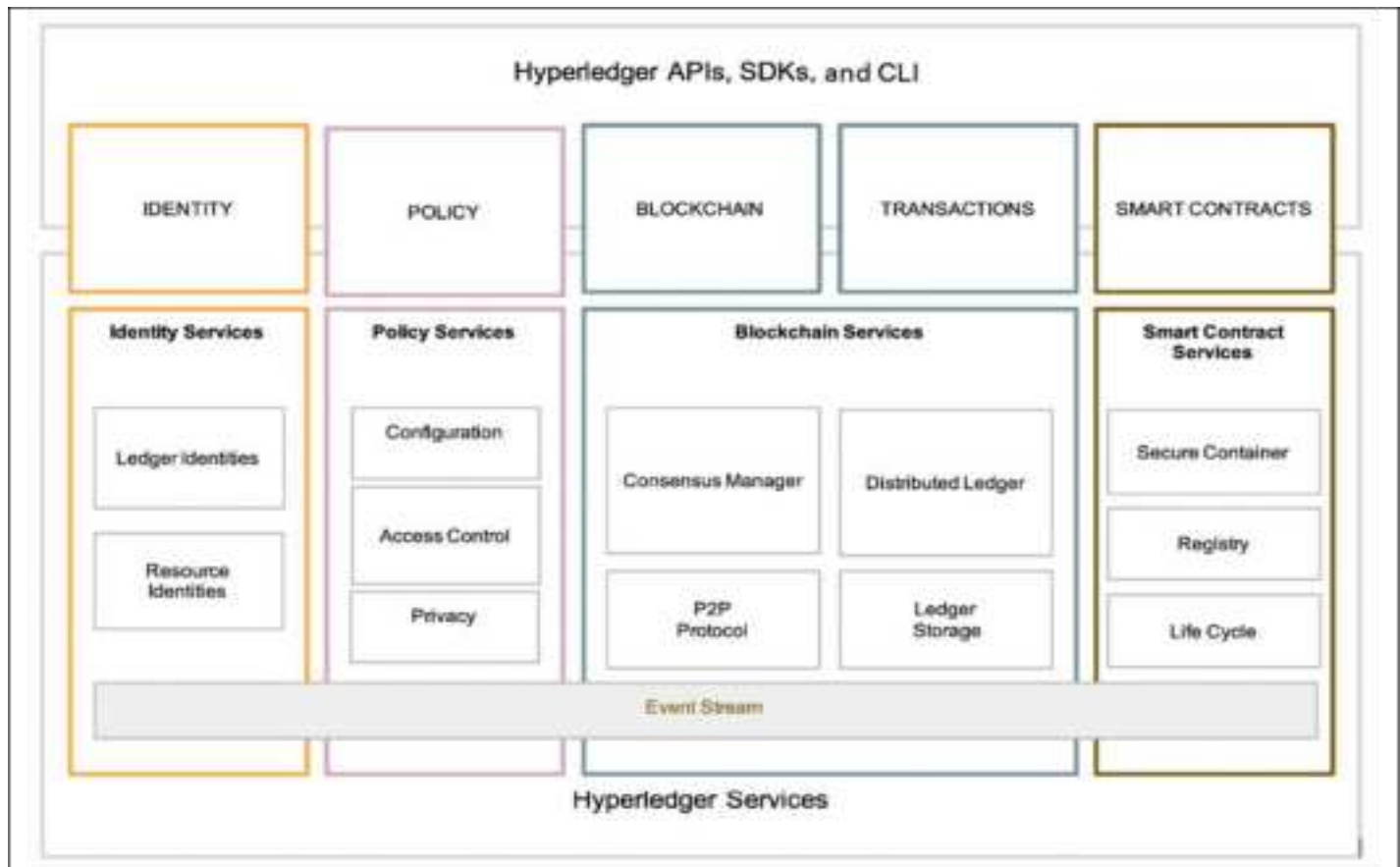


Figure :Reference architecture

In the preceding diagram, starting from the left, we see that we have five top-level components that provide various services. The first is **identity**, which provides authorization, identification, and authentication services under membership services. Then, we have the **policy** component, which provides policy services.

Hyperledger Fabric:

Hyperledger Fabric, or **Fabric** for short, is the contribution made initially by IBM and Digital Assets to the Hyperledger project. This contribution aims to enable a modular, open, and flexible approach toward building blockchain networks.

Various functions in the fabric are pluggable, and it also allows the use of any language to develop smart contracts. This functionality is possible because it is based on container technology (Docker), which can host any language.

Chaincode is sandboxed in a secure container, which includes a secure operating system, the chaincode language, runtime environment, and SDKs for Go, Java, and Node.js. Other languages can be supported too in the future, if required, but this needs some development work. This ability is a compelling feature compared to domain-specific languages in Ethereum, or the limited scripted language in Bitcoin. It is a permissioned network that aims to address issues such as scalability, privacy,...

Hyperledger Sawtooth:

Sawtooth is an enterprise-grade distributed ledger that can run in both permissioned and non-permissioned modes. Sawtooth has several new features, which are introduced in the following sections.

Core feature:

These features include modular design, parallel transaction execution, global state agreement, dynamic consensus, and some other advanced features.

Modular design:

The modular design of Sawtooth enables separation between the application and the core system. This means that developers can focus on the business objectives instead of worrying about the underlying design of the system. The design of Sawtooth can be viewed as a layered architecture where transaction processors manage the application business logic and, on another layer, validators handle the verification and consensus on transactions. A separate layer called the transaction processing layer is responsible for managing transaction...

Setting up a Sawtooth development environment:

There are a few prerequisites that are required in order to set up the development environment.

The easiest way to get Sawtooth up and running is by using Docker. In the following example, we will set up a 5-node network using Docker.

Prerequisites

For this process, you'll need to first install Docker. In this example we are using:

```
$ docker -v
```

```
Docker version 19.03.8, build afacb8b
```

Sawtooth supports different consensus algorithms. In this example we will use PoET. However, other options are available too, such as PBFT. The YAML configuration files for both of these options are available at the links in the following sections.

Using PoET

The following link is available to download the YAML file for setting up Sawtooth with the PoET consensus algorithm: https://sawtooth.hyperledger.org/docs/core/nightly/1-2/app_developers_guide/sawtooth-default-poet.yaml

Using PBFT

Here, you can access the YAML file for setting up Sawtooth with PBFT consensus: https://sawtooth.hyperledger.org/docs/core/nightly/1-2/app_developers_guide/sawtooth-default-pbft.yaml

Setting up a Sawtooth network:

In this , Sawtooth network can be created. First we start with creating a directory and then we will download specific configuration files, which will help with the configuration of the network.

- Create a directory named `sawtooth`:
 - `$ mkdir sawtooth`
- Change the directory to `sawtooth`:
 - `$ cd sawtooth`
- Download the PoET YAML file:

- `$ wget https://sawtooth.hyperledger.org/docs/core/nightly/1-2/app_developers_guide/sawtooth-default-poet.yaml`

This command will show an output like the following, indicating that the `sawtooth-default-poet.yaml` file has been downloaded successfully. Note that only the final part of the output is shown for brevity:

```
...
2020-06-29 21:04:22 (51.9 KB/s) - 'sawtooth-default-poet.yaml' saved [16543/16543]
```

- Start the network:
- `$ docker-compose -f sawtooth-default-poet.yaml up`

This will show a long output and will take several minutes to complete. The output will be similar to the following, which shows the progress of the process:

```
Creating network "sawtooth_default" with the default driver
Creating volume "sawtooth_poet-shared" with default driver
chime: Pulling from hyperledger/sawtooth-intkey-tp-python
```

The Sawtooth Lake Distributed Ledger :

It is a software framework for constructing decentralized ledgers with extensible transaction types. It is comparable to the blockchain ledger that underlies Bitcoin. Sawtooth Lake uses a unique mechanism for reaching consensus on the validity of the ledger based on trusted code running inside a hardware-protected Intel Software Guard Extensions (SGX) enclave.

One of the initial transaction families supported by Sawtooth Lake is the MarketPlace. The MarketPlace Transaction Family establishes the concepts of participants, accounts, assets, holdings, liabilities, and offers in a decentralized ledger to facilitate the exchange of digital assets. The Sawtooth Lake architecture allows the definition of additional transaction families or the consumption of an existing asset-type agnostic transaction family (like MarketPlace) to meet domain-specific needs.

sawtooth-core:

Contains fundamental classes used throughout the Sawtooth Lake project, as well as:

- The gossip networking layer
- Basic transaction, block, and message objects
- The base journal implementation
- The PoET journal consensus mechanism
- Built-in transaction families - Endpoint Registry and Integer KeyRegistry
- The implementation of a server, known as the validator
 - acts as a node on the gossip network
 - validators exchange and act upon messages, as defined by the core classes and via additional plug-in transaction families like the MarketPlace Transaction Family
- The MarketPlace Transaction Family, located in the extensions directory.
 - demonstrates how to inherit and extend base sawtooth-core object types to implement a custom transaction family
 - includes a command line interface called *mktclient* for interacting with validators running the MarketPlace Transaction Family
 - useful for buying, selling and trading digital assets

- Example code, in the form of games, which demonstrate key concepts of Sawtooth Lake
- Tools including a Vagrant environment for easily launching a network of validators
- Source files for this documentation

Core Architecture:

The Sawtooth Lake Distributed Ledger consists of three major architectural layers: the Ledger layer, the Journal layer, and the Communication Layer.

Ledgers:

Ledgers are a conceptual semantic and data model layer for transaction types. Ledgers are described as a ‘conceptual’ layer because they are implemented as a specialization of existing base classes already present in the Communication and Journal layers.

In addition to some in-built system ledgers (Endpoint Registry, and Integer Key Registry), implementing new classes in the ledger layer allows for the creation of new transaction families. The MarketPlace Transaction Family, located in the extensions directory of sawtooth-core, is a good example of how the ledger layer can be extended.

Journals

A journal handles consensus on blocks of identifiers. Identifiers reference transactions, which are globally replicated. In order to confirm blocks, nodes need a copy of the transaction. In this fashion, the journal provides global consensus on block ordering, transaction ordering within blocks, and the content of transactions.

The journal module in sawtooth-core contains:

- The Implementation Of The Base Transaction And Transaction Block Classes
- The Consensus Algorithms
- The Global Store Manager
- The Block Store And Key Value Store

The consensus journal object is `journal.journal_core.Journal` in the sawtooth-core repository.

Consensus Mechanisms:

Sawtooth Lake implements PoET as a consensus mechanism.

PoET and SGX:

The Sawtooth Lake Distributed Ledger provides a unique mechanism to ensure fairness in the node lottery. Instead of a Proof-of-Work competition amongst nodes, Sawtooth Lake implements a Proof-of-Elapsed-Time (PoET) algorithm for distributed consensus. PoET relies upon a trusted execution environment, Intel's Software Guard Extensions (SGX), to generate fair, verifiable random wait timers and signed certificates of timer expiration. This mechanism substantially reduces the computation and energy cost of ensuring fair distributed consensus.

The implementation of PoET in Sawtooth Lake runs in a simulated enclave, not a true trusted execution environment. For this reason, attestation that wait timers have been fairly generated is not possible. This version of PoET is intended for experimental purposes and should not be used as the consensus mechanism in any 'production' environment.